CS2109S: Introduction to AI and Machine Learning

Lecture 9:
Convolutional Neural Networks

24 March 2026

PollEv.com/conghuihu365

Annoucement

Tutorial 8 (Next week)

- Schedule affected by NUS Well-Being Day and Good Friday.
- Similar to the CNY arrangement, many tutorial sessions will be pre-recorded.
- 1000 Free EXP
- More details can be found here

Outline

- Issues of Using FCNN
- Convolution
  - Convolution operation
  - Convolution with padding
  - Multi-channel convolution
- Convolutional Neural Network
  - Convolutional Layer
  - Pooling Layer
  - Fully-connected Layer

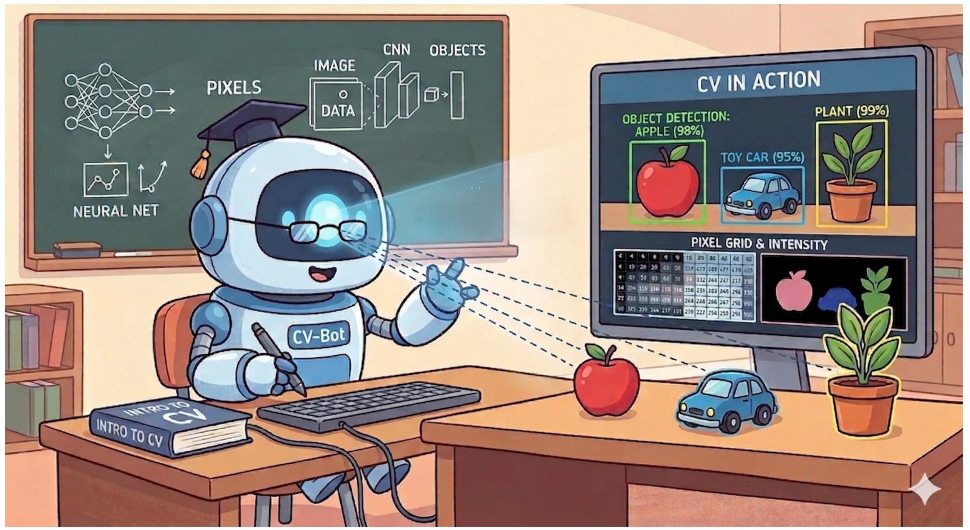
What is Computer Vision

A field of Artificial Intelligence

- Empower computers to extract meaningful information from visual inputs
- Give machines the ability to “see” and interpret the visual world.

Visual inputs:

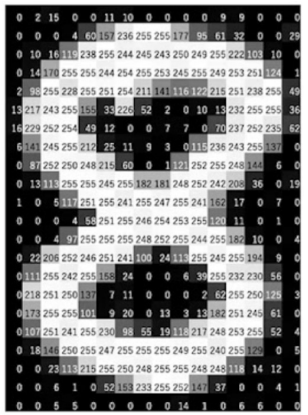
- **Image**
- Video
- 3D Point Cloud
- …



How Computers See Images

Grayscale Image:

- Represented as a 2D grid of pixels.
- Every pixel is a single intensity value indicating brightness.
- Pixel values range from 0 (Black) to 255 (White)
- Image shape: Height (number of rows) x Width (number of columns)



How Computers See Images

Colour (RGB) image

- Represented as a 2D grid of pixels with three colour channels
- Each pixel contains three values representing: Red (R), Green (G), Blue (B)
- Pixel values range from 0 to 255
- Image shape: Height (number of rows) × Width (number of columns) × 3

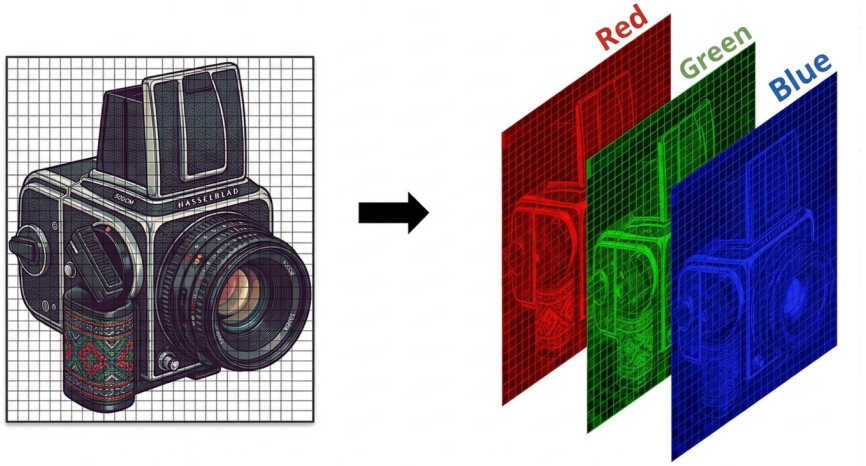


Image Classification

Image classification is a fundamental task in computer vision:

- Given an image, predict what object it contains.

- **Example: Grayscale image binary classification**
  - Task: Decide whether the grayscale image contains a cat.

Cat or **not**?

Data

Suppose:

- We are given n data points.
- Each data point consists of a grayscale image and a target label.
- Each image is a 2D grid of pixels.
- The target is 0 (“no”) or 1 (“yes”).

Data – Math

Suppose

$$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\},$$

where for all $i \in \{1, \dots, n\}$

Features: $x^{(i)} \in \mathbb{R}^{I_H \times I_W}$

Targets: $y^{(i)} \in \mathbb{R}$

Fully-connected (FC) NN Approach

A FCNN only accepts feature vector $\in \mathbb{R}^d$ as input.

- Images are naturally 2D grids of pixels
- We must flatten the image by unrolling all pixels into a single vector.

Example:

- Image shape: 600×600
- After flattening, the input becomes a vector with $\mathbf{360,000}$ **values**.

Fully-connected (FC) NN Approach

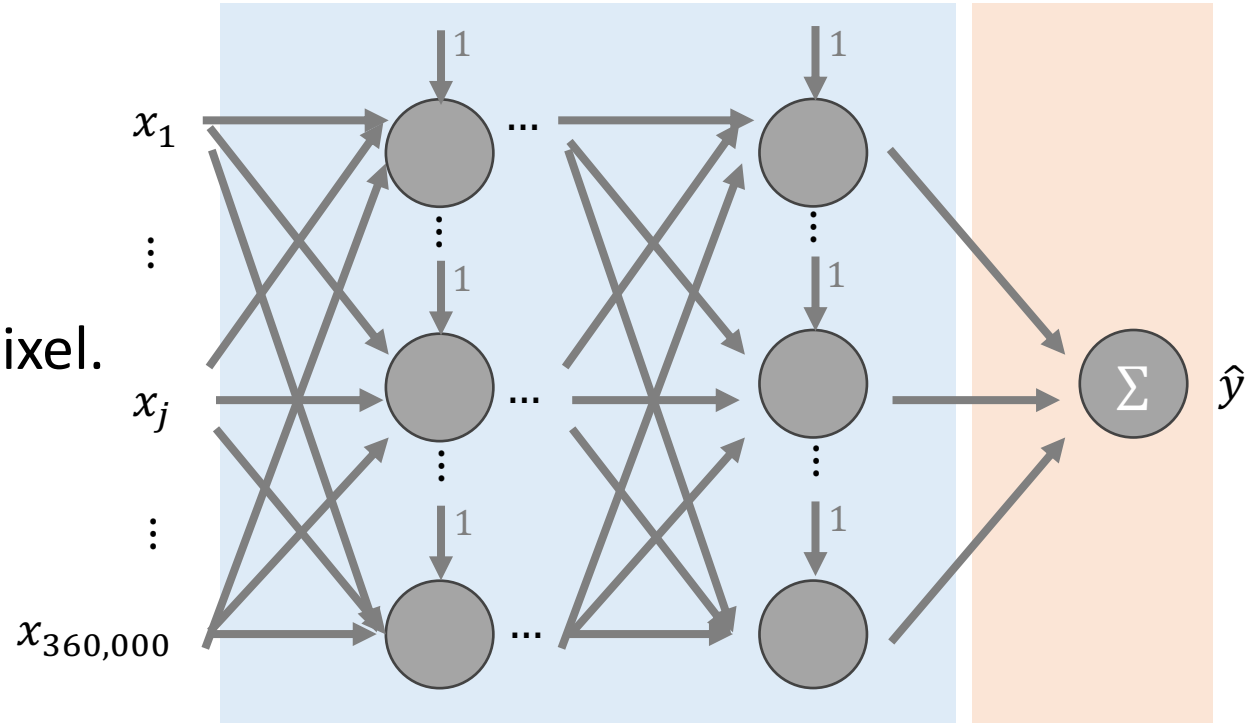
Given an image represented as a vector $x \in \mathbb{R}^{360,000}$, predict whether the image contains a cat, i.e., $y \in \{0,1\}$.

Assume:

- First hidden layer has 1000 neurons
- Each neuron connects to every input pixel.

Learnable parameters in first layer:

$$\underbrace{360{,}000 \times 1000}_{\text{Weights}} + \underbrace{1000}_{\text{Bias}} = 360{,}001{,}000$$



Issues of Using FCNN

By flattening the image, the FCNN evaluates a massive 1D vector

☹ **Parameter Explosion**

- Looking at the whole vector at once
- Very large memory requirement
- Risk of overfitting
- Slow training

☹ **Loss of Spatial Structure**

- Spatial structure carries important visual information.
- Edges and shapes depend entirely on neighboring pixels.
- Flattening destroys this crucial proximity

The Paradigm Shift

To solve the image classification task, we need a new architecture that:

🎯 Parameter Efficiency

- Reduce the massive number of learnable parameters
- Prevent memory exhaustion and overfitting.

🎯 2D Processing

- Process images natively in their original 2D grid structure
- Preserve spatial proximity.

Solution: A Convolutional Neural Network that uses Convolution Operation.

Outline

- FCNN for Vision Task
- **Convolution**
  - Convolution operation
  - Convolution with padding
  - Multi-channel convolution
- Convolutional Neural Network
  - Convolutional Layer
  - Pooling Layer
  - Fully-connected Layer

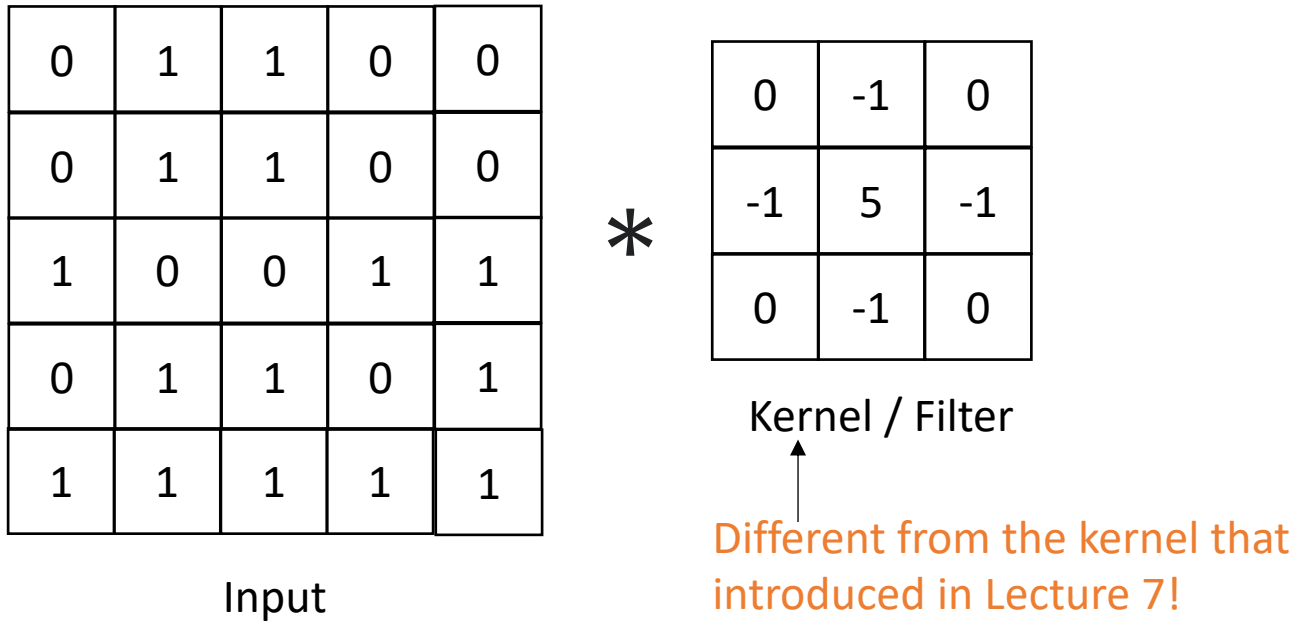
Convolution Operation

Components:

Input: The data being processed, e.g., a grid of pixels.

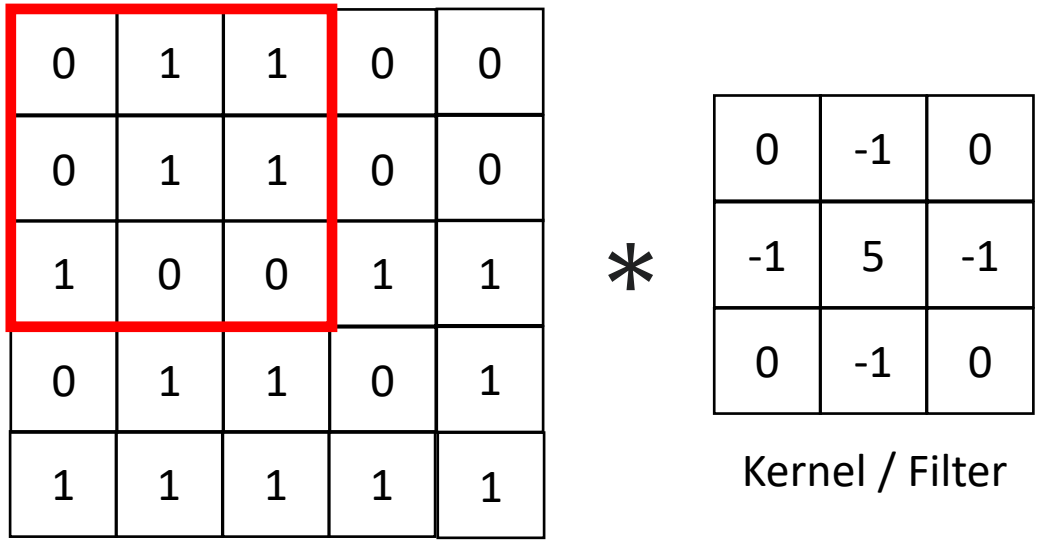
Kernel (Filter): A small grid of weights

Operation: Represented using the symbol *.



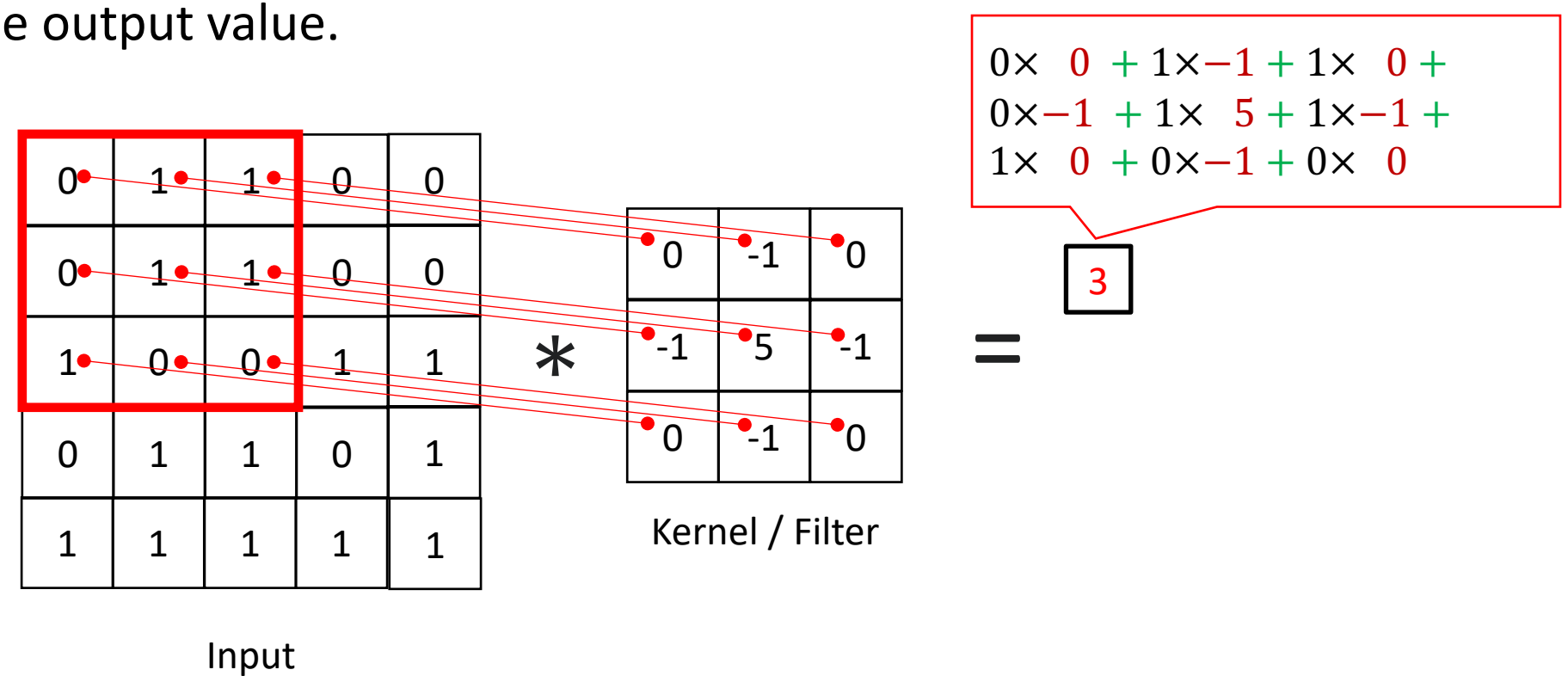
Convolution Operation: 2D

Step 1: Overlay the Kernel: Position the kernel over a patch of the input data, starting from the top-left corner.



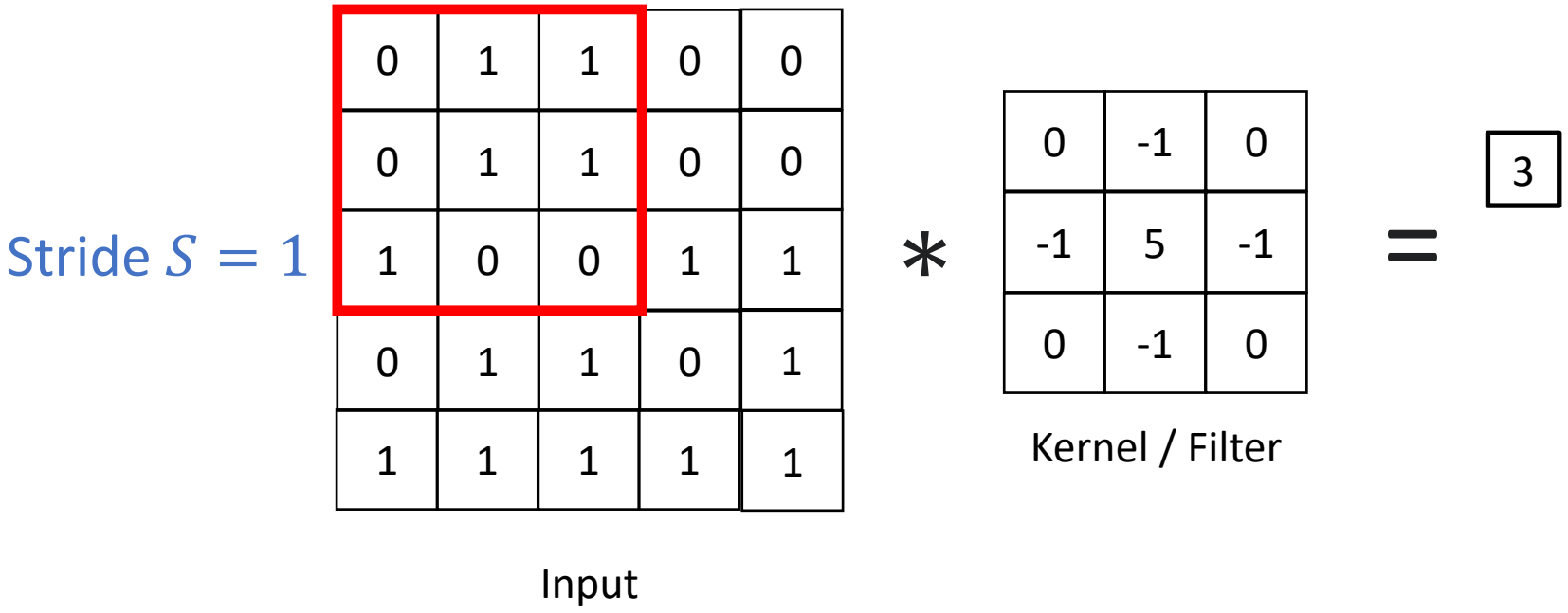
Convolution Operation: 2D

Step 2: Compute the Weighted Sum: Perform an element-wise multiplication between the kernel's values and the corresponding values in the input patch. Sum all the products to get a single output value.



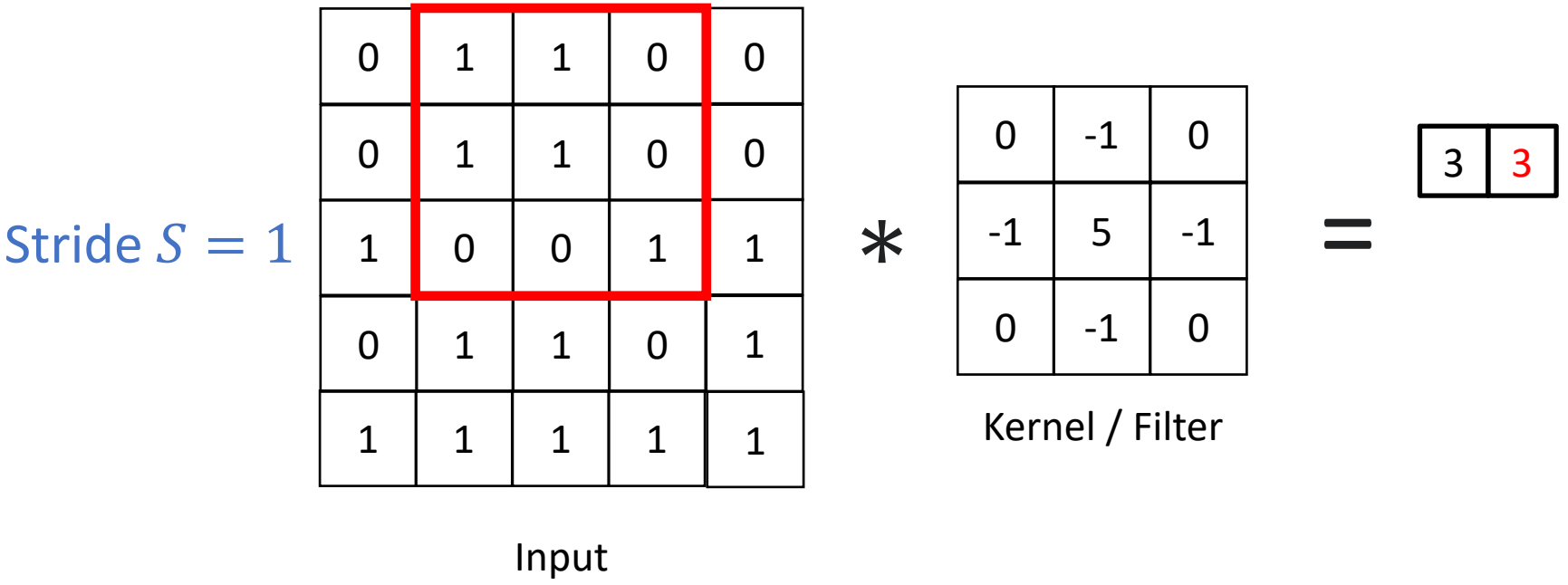
Convolution Operation: 2D

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input.



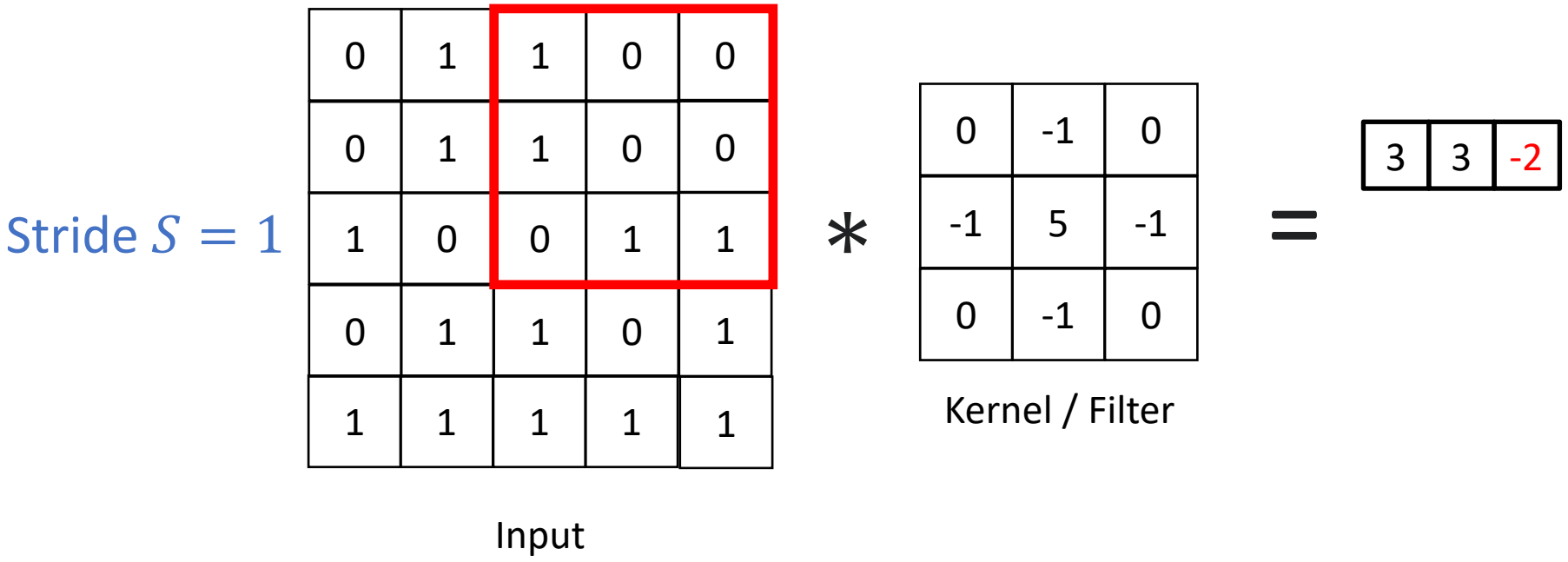
Convolution Operation: 2D

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input.



Convolution Operation: 2D

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input.

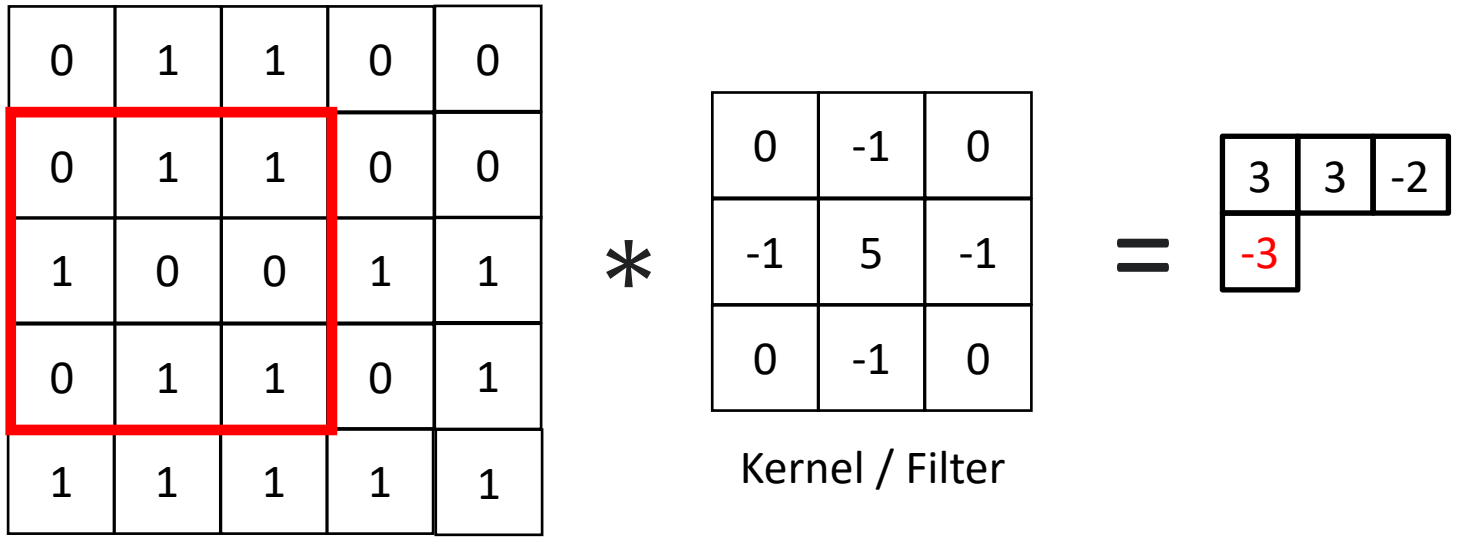


Convolution Operation: 2D

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input.

When horizontal movement is no longer possible, move downward by the step size.

Stride $S = 1$



Input

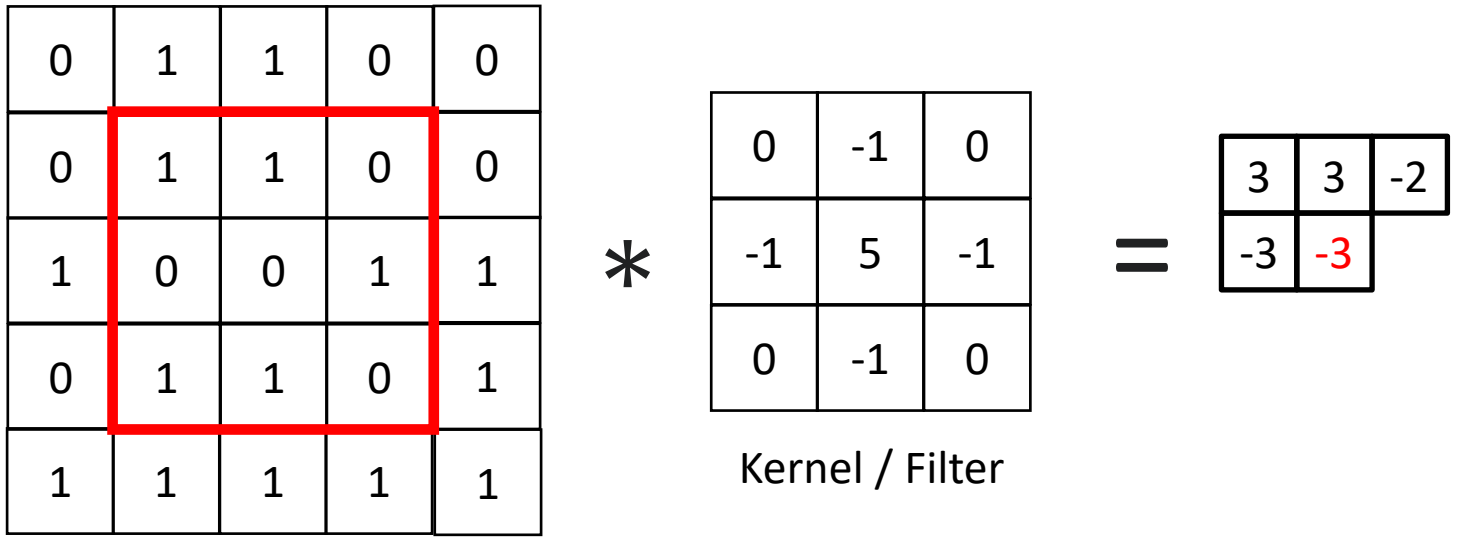
Kernel / Filter

Convolution Operation: 2D

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input.

When horizontal movement is no longer possible, move downward by the step size.

Stride $S = 1$

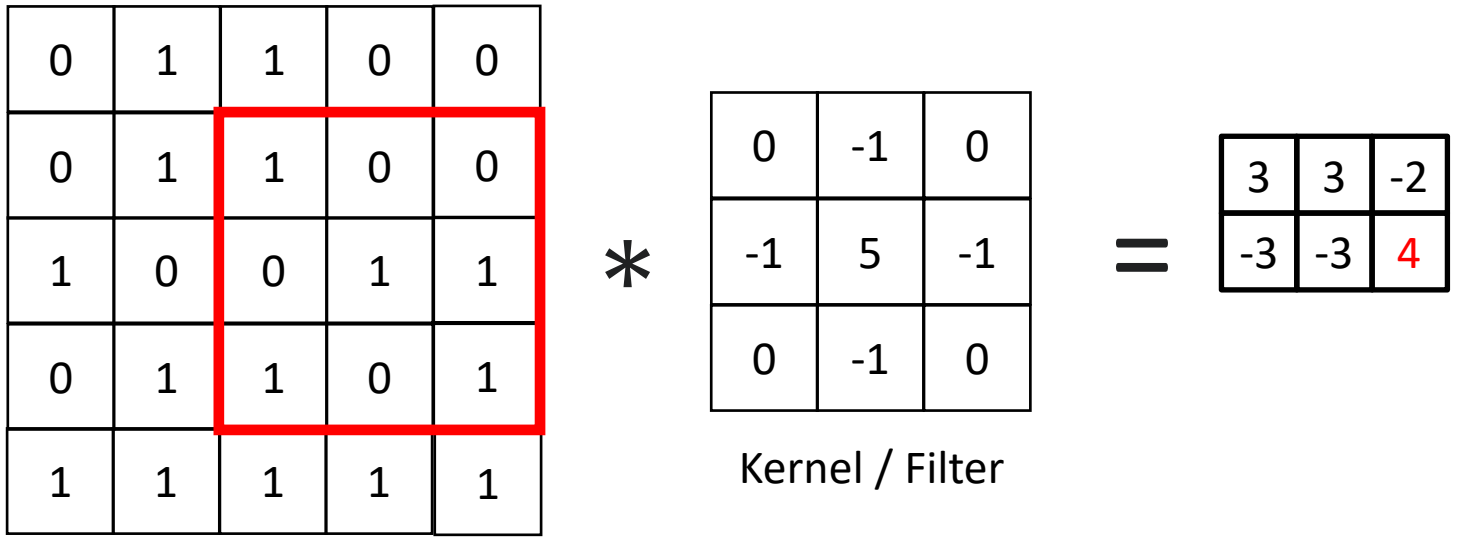


Convolution Operation: 2D

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input.

When horizontal movement is no longer possible, move downward by the step size.

Stride $S = 1$

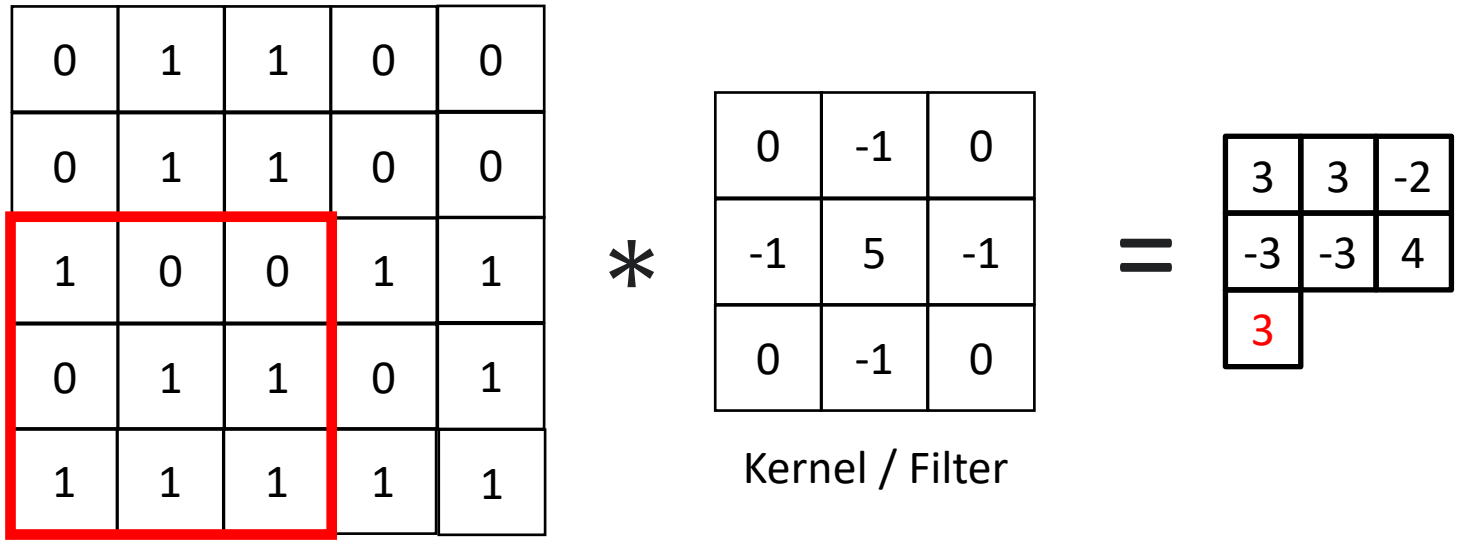


Convolution Operation: 2D

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input.

When horizontal movement is no longer possible, move downward by the step size.

Stride $S = 1$

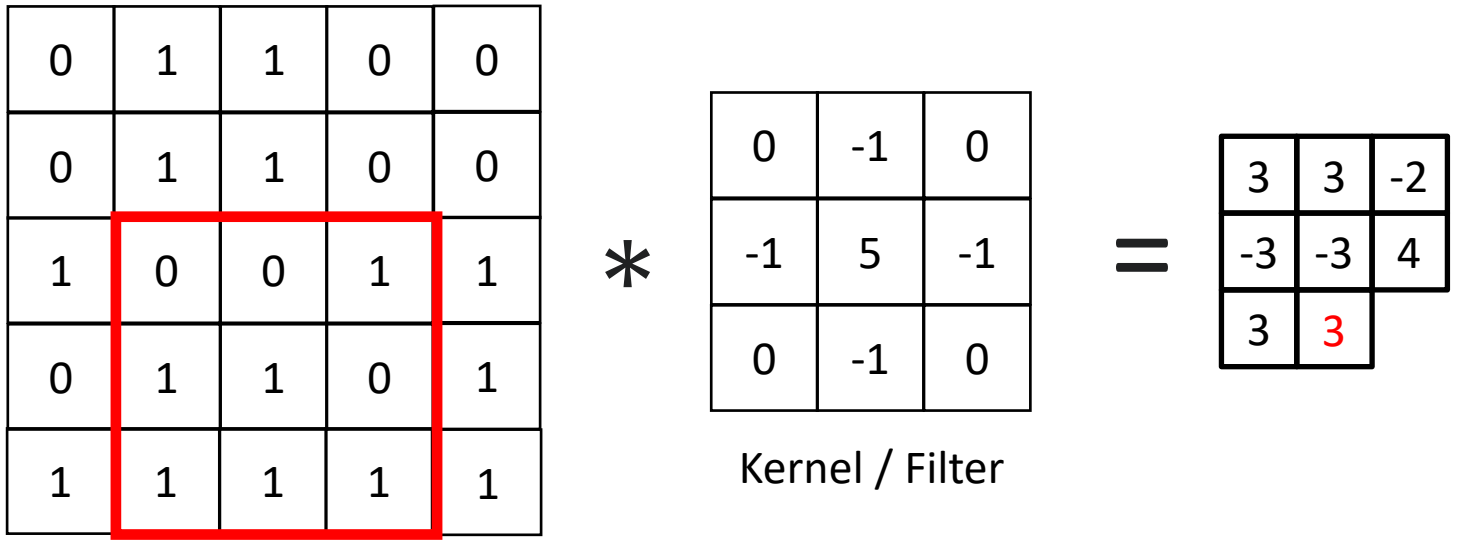


Convolution Operation: 2D

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input.

When horizontal movement is no longer possible, move downward by the step size.

Stride $S = 1$

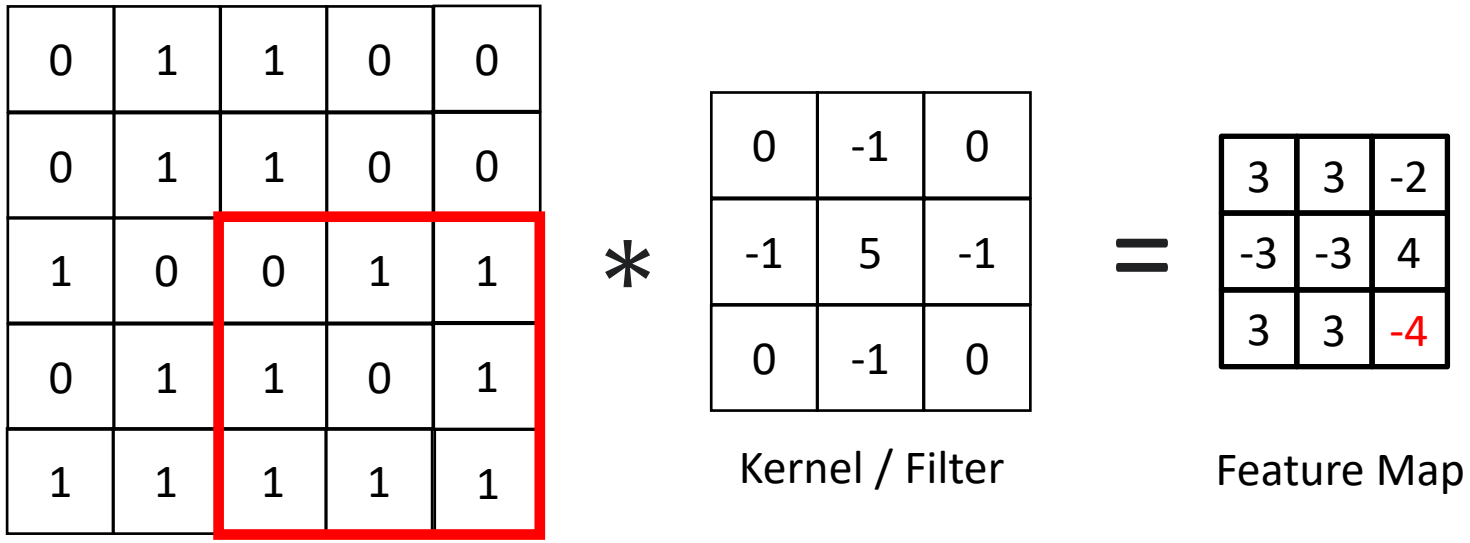


Convolution Operation: 2D

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input.

When horizontal movement is no longer possible, move downward by the step size.

Stride $S = 1$



Input

Kernel / Filter

Feature Map

Stride

Stride (S) is the step size the kernel takes as it slides across the input matrix.

The Default ($S = 1$): The kernel moves one pixel at a time.

Increasing the Stride ($S > 1$): Reduce the height and width of the output feature map.

Stride $S = 2$

0	1	1	0	0
0	1	1	0	0
1	0	0	1	1
0	1	1	0	1
1	1	1	1	1

Input

$*$

0	-1	0
-1	5	-1
0	-1	0

Kernel / Filter

$=$

3

Feature Map

Stride

Stride (S) is the step size the kernel takes as it slides across the input matrix.

The Default ($S = 1$): The kernel moves one pixel at a time.

Increasing the Stride ($S > 1$): Reduce the height and width of the output feature map.

Stride $S = 2$

0	1	1	0	0
0	1	1	0	0
1	0	0	1	1
0	1	1	0	1
1	1	1	1	1

Input

$*$

0	-1	0
-1	5	-1
0	-1	0

Kernel / Filter

$=$

3	-2

Feature Map

Stride

Stride (S) is the step size the kernel takes as it slides across the input matrix.

The Default ($S = 1$): The kernel moves one pixel at a time.

Increasing the Stride ($S > 1$): Reduce the height and width of the output feature map.

Stride $S = 2$

0	1	1	0	0
0	1	1	0	0
1	0	0	1	1
0	1	1	0	1
1	1	1	1	1

Input

$*$

0	-1	0
-1	5	-1
0	-1	0

Kernel / Filter

$=$

3	-2
3	

Feature Map

Stride

Stride (S) is the step size the kernel takes as it slides across the input matrix.

The Default ($S = 1$): The kernel moves one pixel at a time.

Increasing the Stride ($S > 1$): Reduce the height and width of the output feature map.

Stride $S = 2$

0	1	1	0	0
0	1	1	0	0
1	0	0	1	1
0	1	1	0	1
1	1	1	1	1

Input

$*$

0	-1	0
-1	5	-1
0	-1	0

Kernel / Filter

$=$

3	-2
3	-4

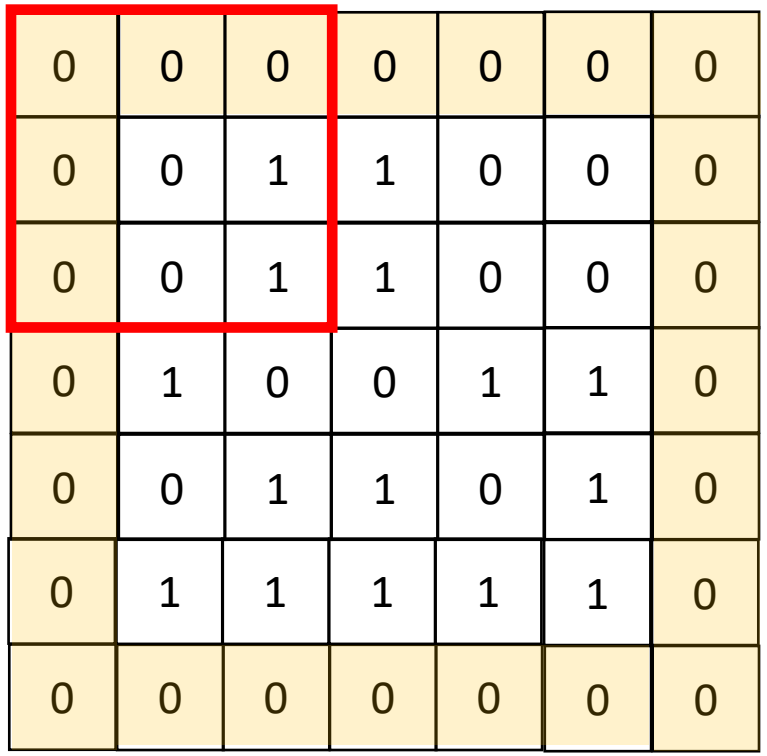
Feature Map

Convolution with Padding

Padding adds extra pixels around the input border, usually filling them with zeros.

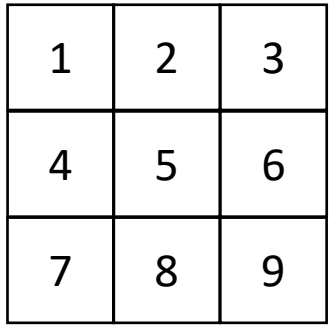
Padding $P = 1$

Stride $S = 1$



Input
Shape before padding: 5 x 5

$*$



Kernel / Filter

$=$

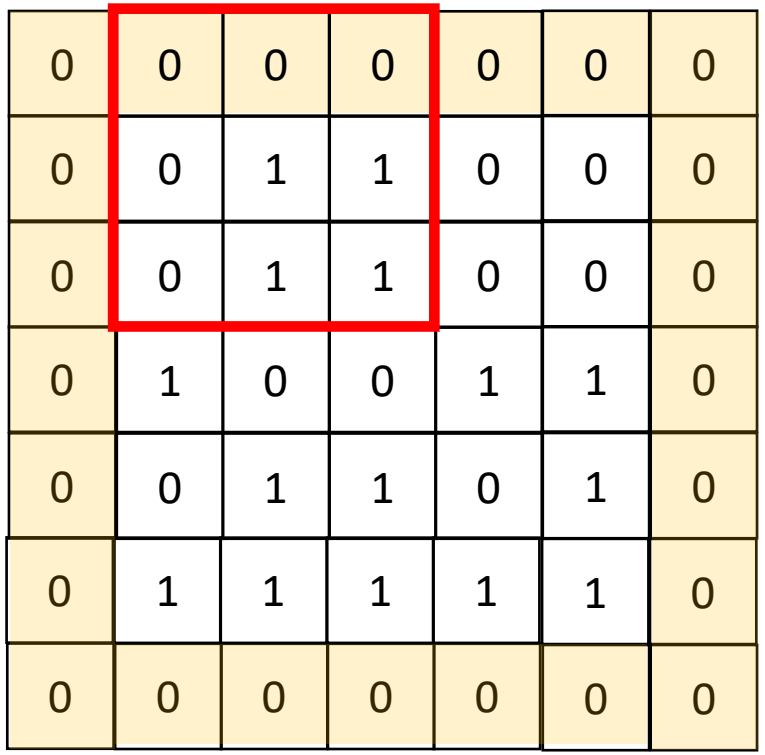
Feature map

Convolution with Padding

Padding adds extra pixels around the input border, usually filling them with zeros.

Padding $P = 1$

Stride $S = 1$



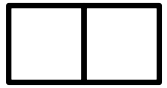
Input
Shape before padding: 5 x 5

$*$

1	2	3
4	5	6
7	8	9

Kernel / Filter

$=$



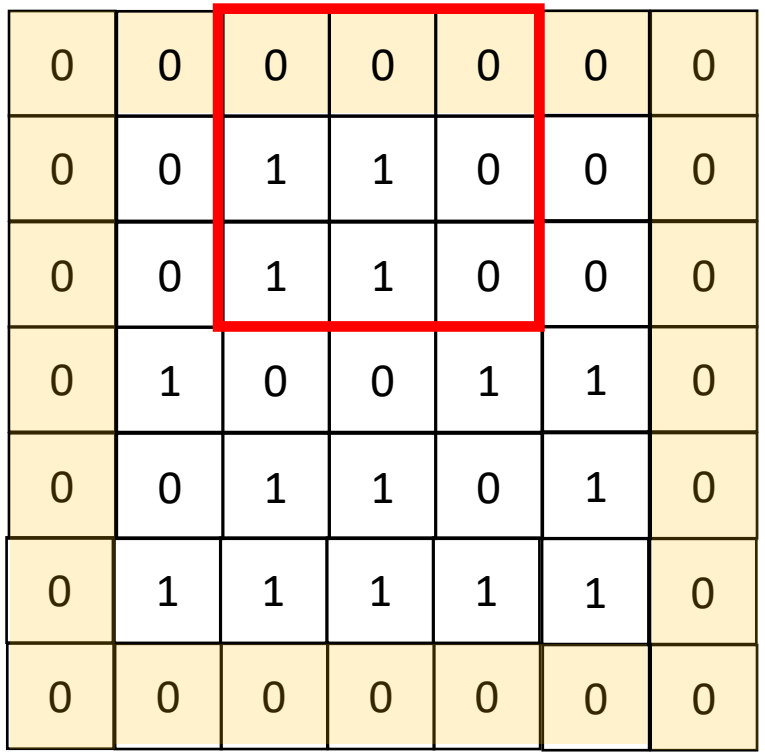
Feature map

Convolution with Padding

Padding adds extra pixels around the input border, usually filling them with zeros.

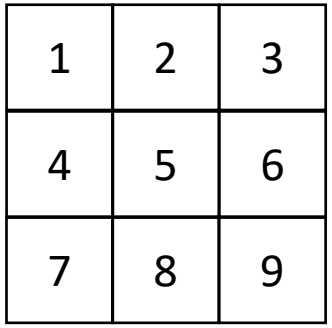
Padding $P = 1$

Stride $S = 1$



Input
Shape before padding: 5 x 5

$*$



Kernel / Filter

$=$

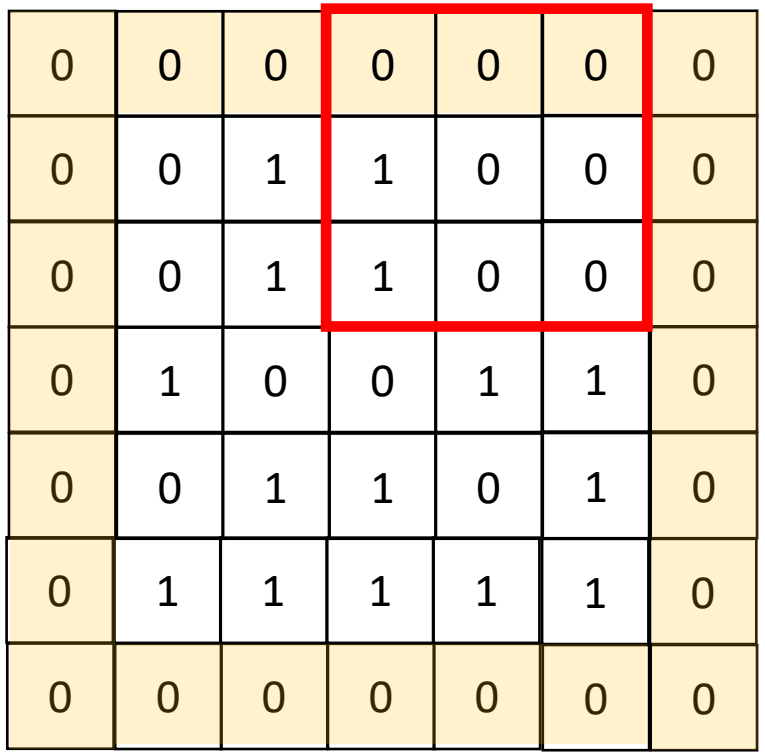
Feature map

Convolution with Padding

Padding adds extra pixels around the input border, usually filling them with zeros.

Padding $P = 1$

Stride $S = 1$



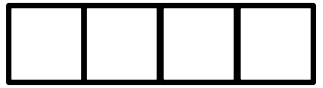
Input
Shape before padding: 5 x 5

$*$

1	2	3
4	5	6
7	8	9

Kernel / Filter

$=$



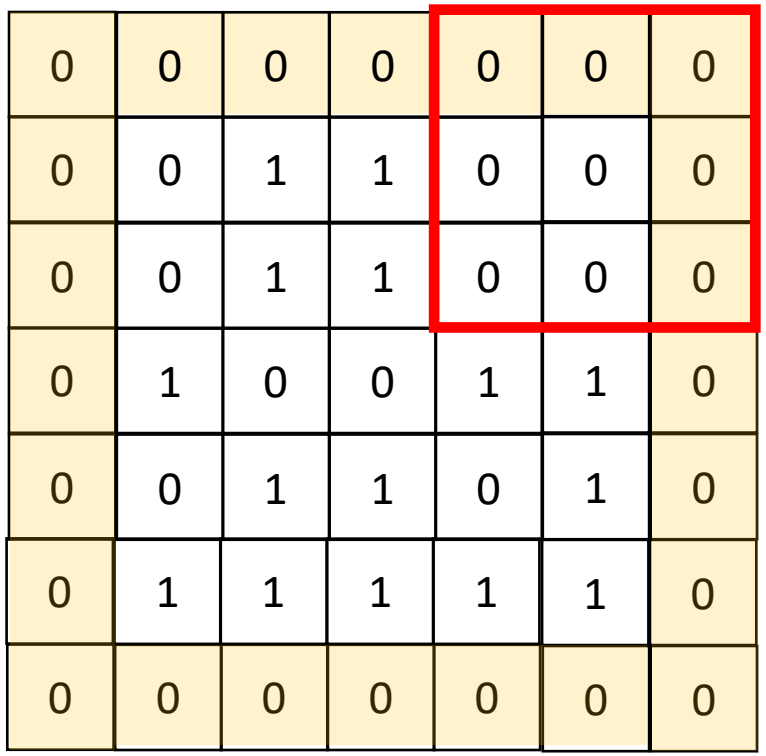
Feature map

Convolution with Padding

Padding adds extra pixels around the input border, usually filling them with zeros.

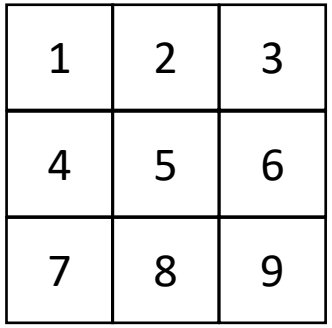
Padding $P = 1$

Stride $S = 1$



Input
Shape before padding: 5 x 5

$*$



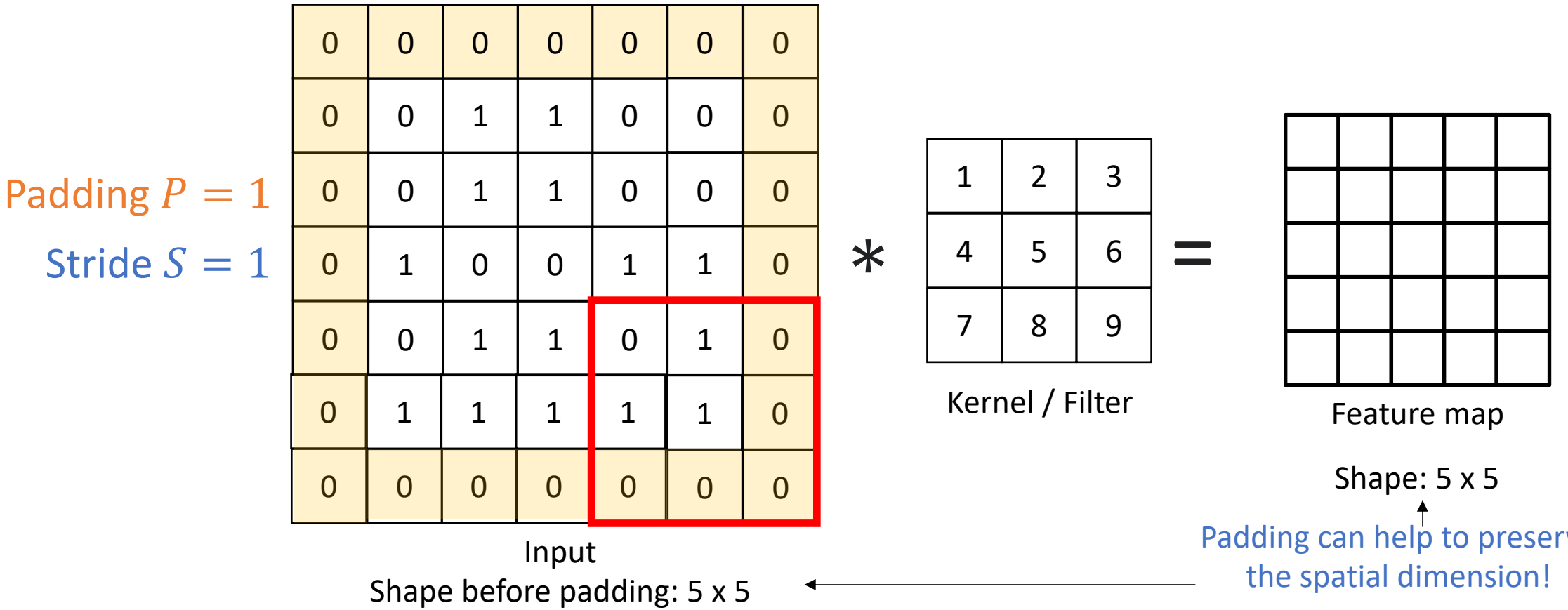
Kernel / Filter

$=$

Feature map

Convolution with Padding

Padding adds extra pixels around the input border, usually filling them with zeros.



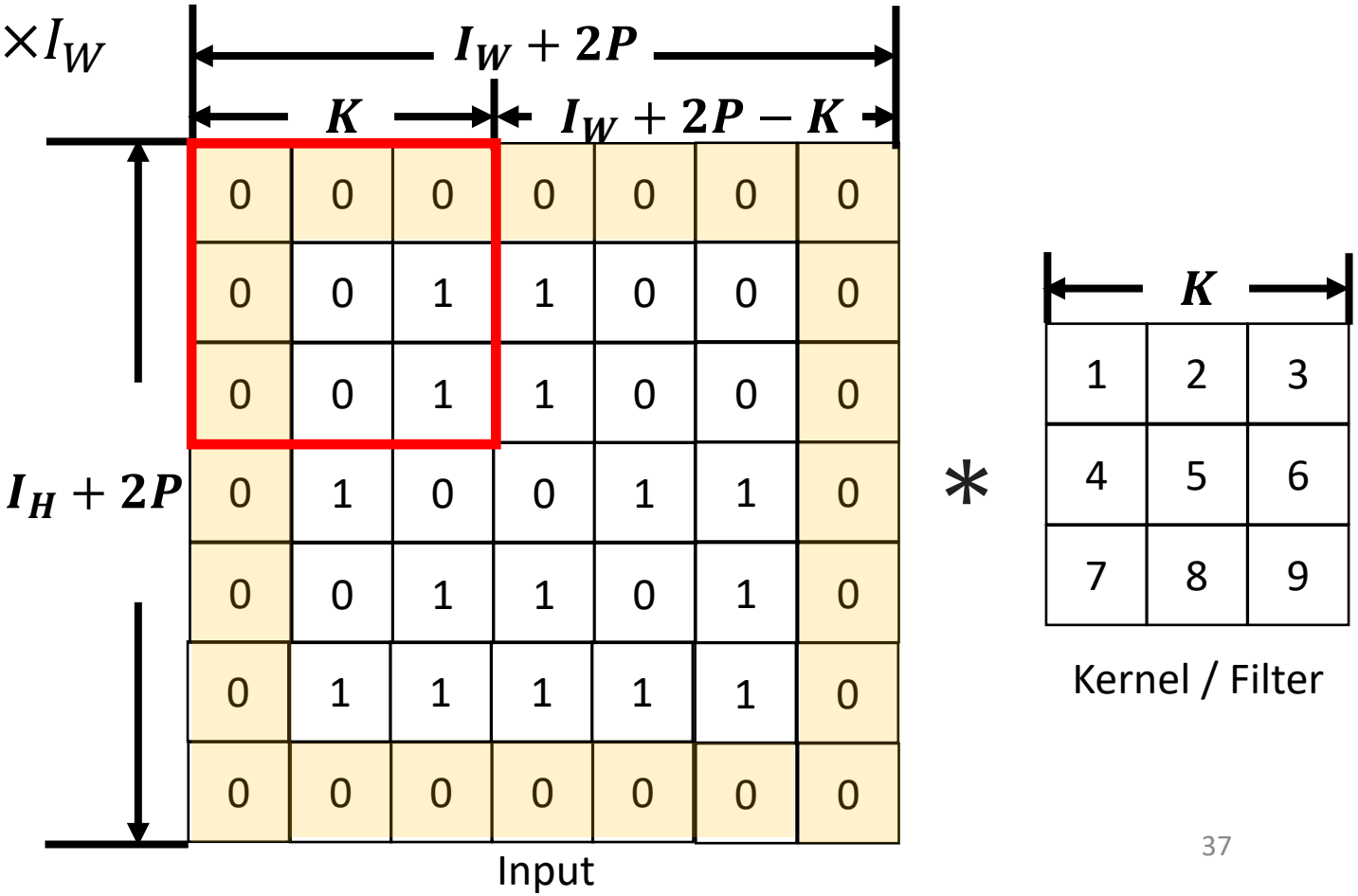
Convolution: Output Shape

The setup:

- Input shape before padding: $I_H \times I_W$
- Kernel shape: $K \times K$
- Padding: P
- Stride: S

Output Width: $1 + \left\lfloor \frac{I_W + 2P - K}{S} \right\rfloor$

Output Height: $1 + \left\lfloor \frac{I_H + 2P - K}{S} \right\rfloor$



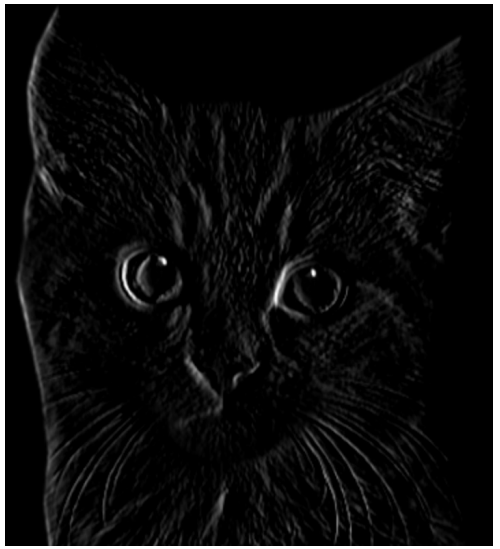
Kernel Weights

- Kernel weights determine what feature is detected.
- Different weights → different extracted features from the same input.

Original

Horizontal Edge Detector

1	1	1
0	0	0
-1	-1	-1



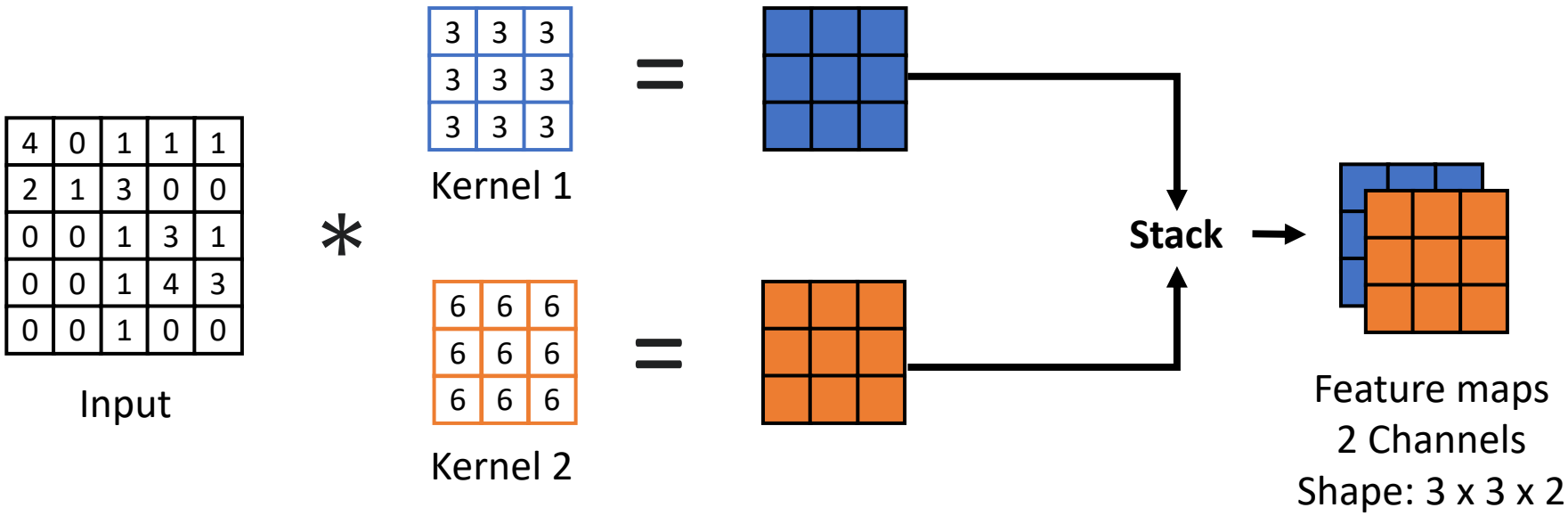
Vertical Edge Detector

1	0	-1
1	0	-1
1	0	-1

Try it yourself: https://setosa.io/ev/image-kernels/

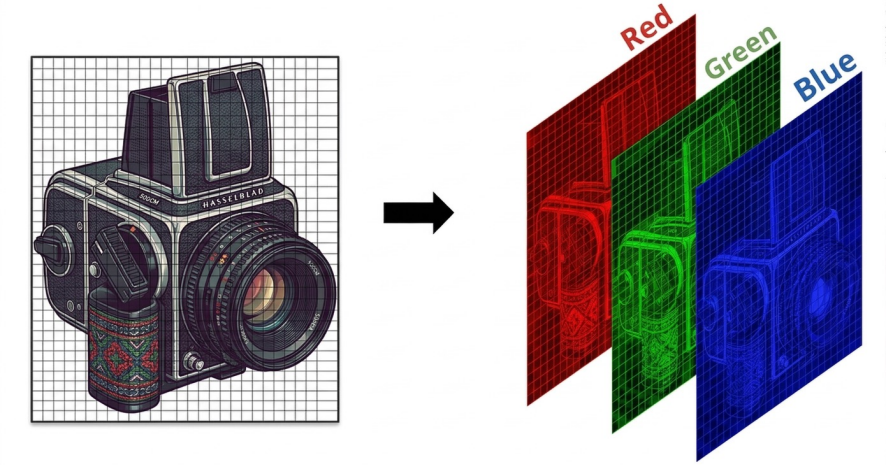
Convolution Operation: Multiple Kernels

- Each kernel can detect a specific type of feature, e.g, edge.
- Real-world images contain many different features.
- Therefore, multiple kernels are applied simultaneously to extract different features.



Convolution Operation: Multi-Channel Inputs

- So far, we have considered inputs with a single channel, with shape Height × Width.
- However, colour images are represented using RGB (Red, Green, Blue) channels.
- Therefore, an input can have multiple channels, with shape Height × Width × Channels.



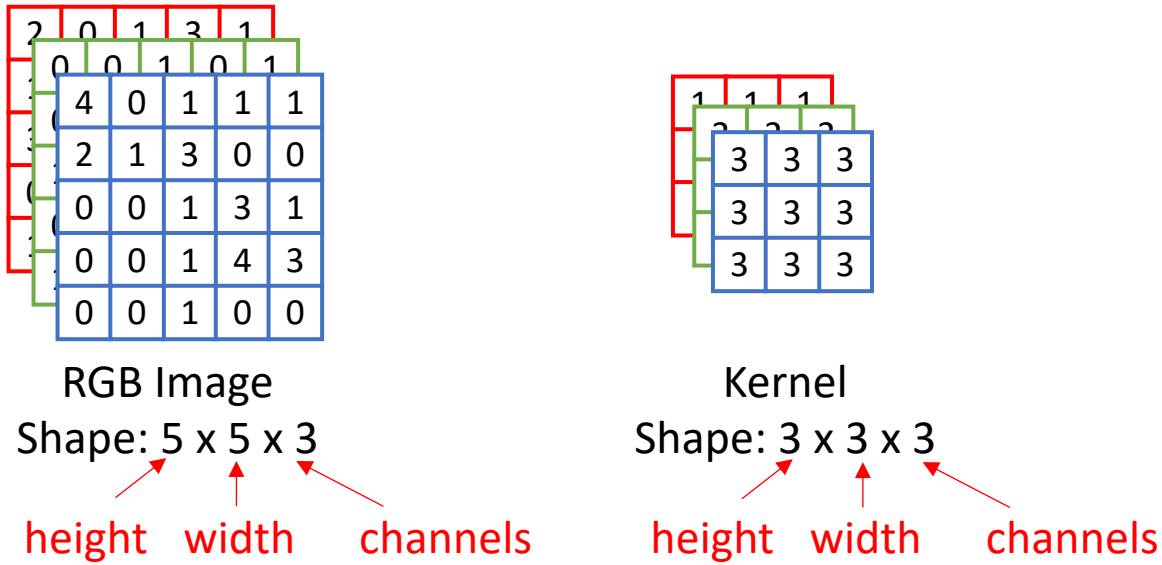
Convolution Operation: Multi-Channel Inputs

- The number of channels in each kernel must match the number of input channels.
- **Channel-specific weights:** Each input channel has its own set of kernel weights.

Example:

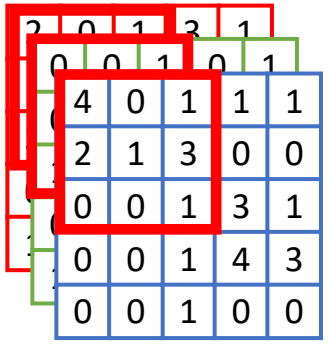
A 3-channel kernel contains 27 weights (3×3×3):

- 9 weights operate on the Red channel
- 9 weights operate on the Green channel
- 9 weights operate on the Blue channel



Convolution Operation: Multi-Channel Inputs

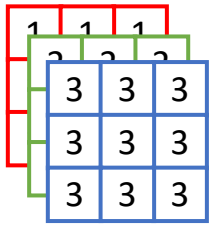
Step 1: Overlay the Kernel: Position the kernel over a patch of the input data, starting from the top-left corner.



Input
3 Channels

Shape: 5 x 5 x 3

*

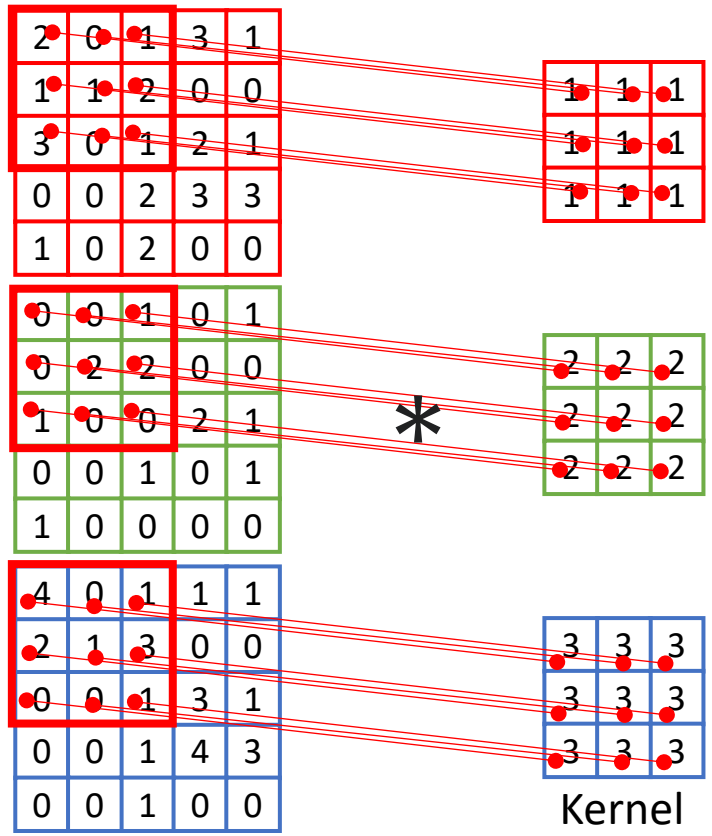


Kernel
3 Channels

Shape: 3 x 3 x 3

Convolution Operation: Multi-Channel Inputs

Step 2: Compute the Weighted Sum: Perform an element-wise multiplication between the kernel's values and the corresponding values in the input patch. Sum all the products to get a single output value.



Input
3 Channels

Stride $S = 1$

$*$

$2 \times 1 + 0 \times 1 + 1 \times 1 +$
$1 \times 1 + 1 \times 1 + 2 \times 1 +$
$3 \times 1 + 0 \times 1 + 1 \times 1$
$= 11$

$0 \times 2 + 0 \times 2 + 1 \times 2 +$
$0 \times 2 + 2 \times 2 + 2 \times 2 +$
$1 \times 2 + 0 \times 2 + 0 \times 2$
$= 12$

$4 \times 3 + 0 \times 3 + 1 \times 3 +$
$2 \times 3 + 1 \times 3 + 3 \times 3 +$
$0 \times 3 + 0 \times 3 + 1 \times 3$
$= 36$

$=$

$11 + 12 + 36$

59

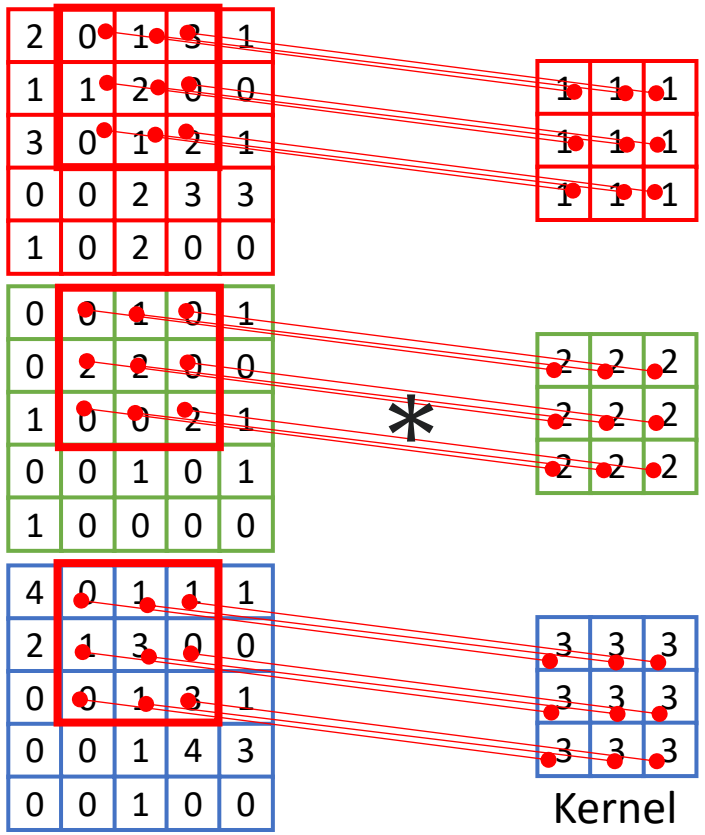
Feature map

Convolution Operation: Multi-Channel Inputs

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input. When horizontal movement is no longer possible, move downward by the step size.

Input
3 Channels

Stride $S = 1$



$*$

$=$

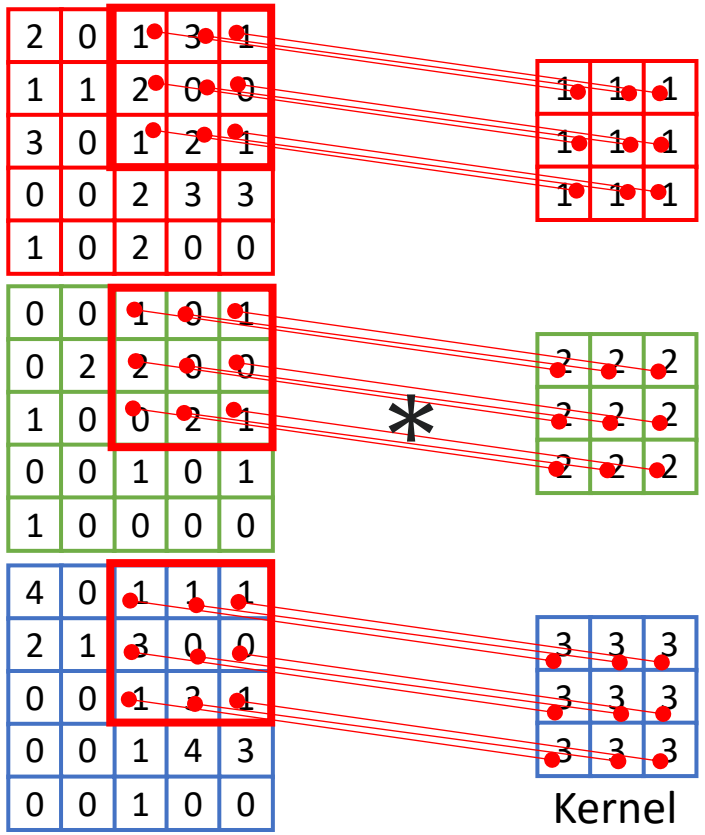
Feature map

Convolution Operation: Multi-Channel Inputs

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input. When horizontal movement is no longer possible, move downward by the step size.

Input
3 Channels

Stride $S = 1$



*

=

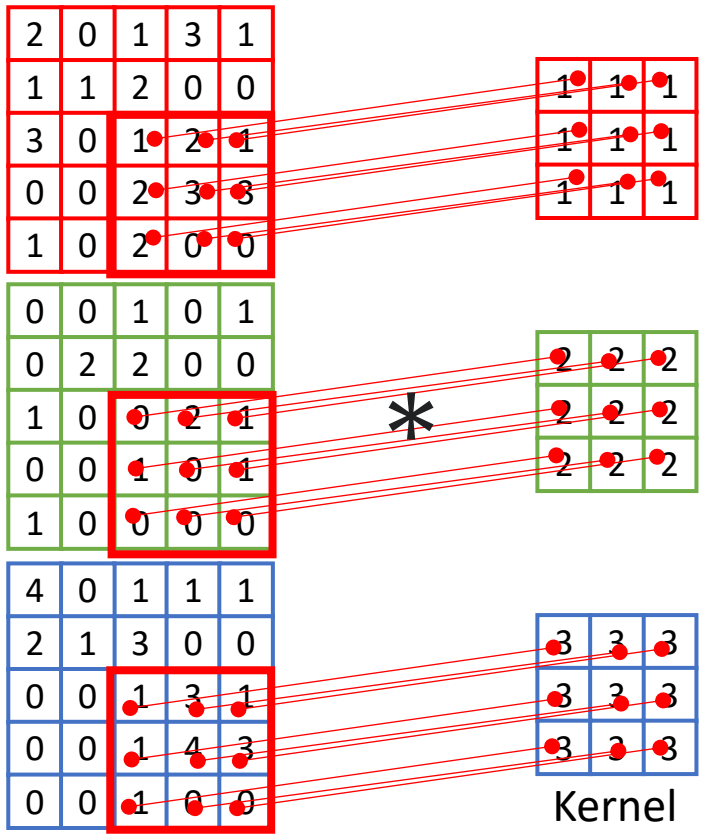
Feature map

Convolution Operation: Multi-Channel Inputs

Slide and Repeat Step 2: Move across the input with step size (Stride), sliding left to right. Repeat Step 2 wherever the kernel fully overlaps with the input. When horizontal movement is no longer possible, move downward by the step size.

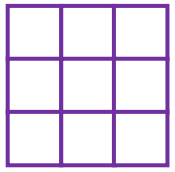
Input
3 Channels

Stride $S = 1$



$*$

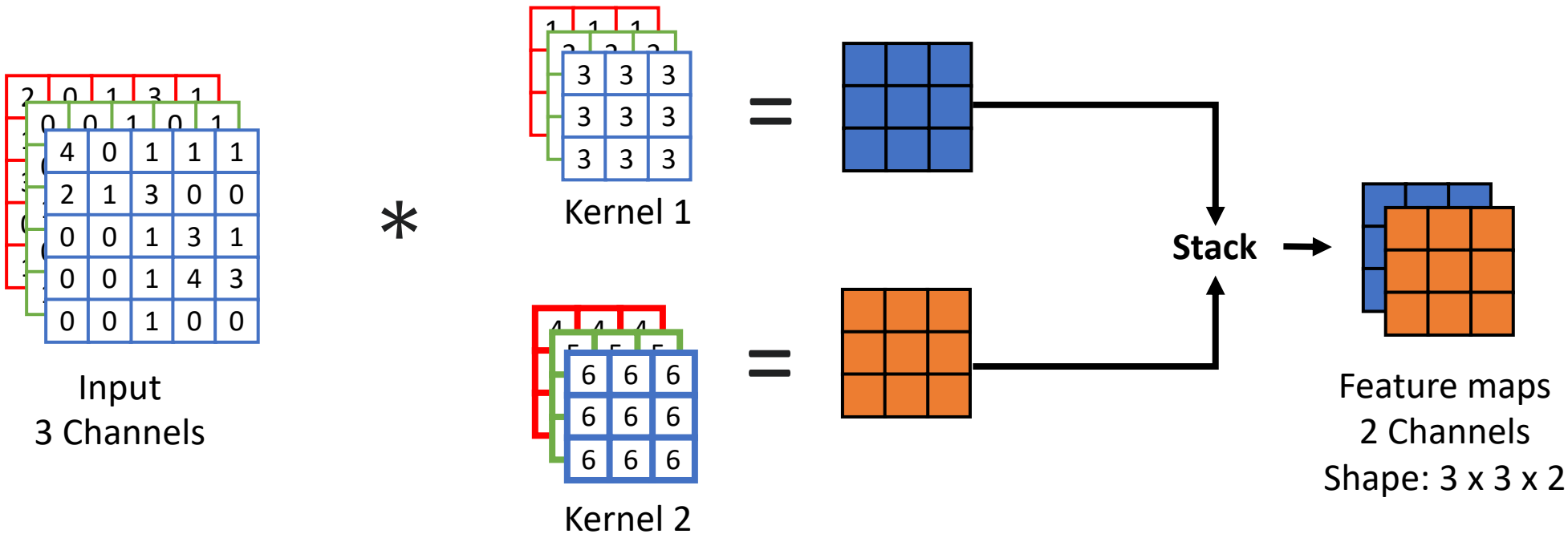
$=$



Feature map

Convolution Operation: Multi-Channel Inputs

- The number of channels in each kernel must match the number of input channels.
- Each kernel generates one feature map.
- The number of kernels determines the number of feature maps (output channels).



Break

HOW DO SELF-DRIVING CARS "SEE"?

Outline

- FCNN for Vision Task
- Convolution
  - Convolution operation
  - Convolution with padding
  - Multi-channel convolution
- **Convolutional Neural Network**
  - Convolutional Layer
  - Pooling Layer
  - Fully-connected Layer

Convolutional Neural Networks

Convolutional Neural Networks (CNNs)

- Extract features using convolution operation.
- Parameters (e.g., weights in kernel) are learned with backpropagation
- Comprehensive discussion of CNN backpropagation is beyond the scope of the course.

Main components:

- Convolutional layers
- Pooling layers
- Fully-connected layers

Convolutional Layer: Pre-activation

Core operation: Apply a set of learnable kernels to an input to extract features via convolution operation.

Bias term: Learnable scalar(s) added to the convolution output.

- One bias per output feature map

- Before activation, the ***i*-th Output Channel** (Z_i) is computed as:

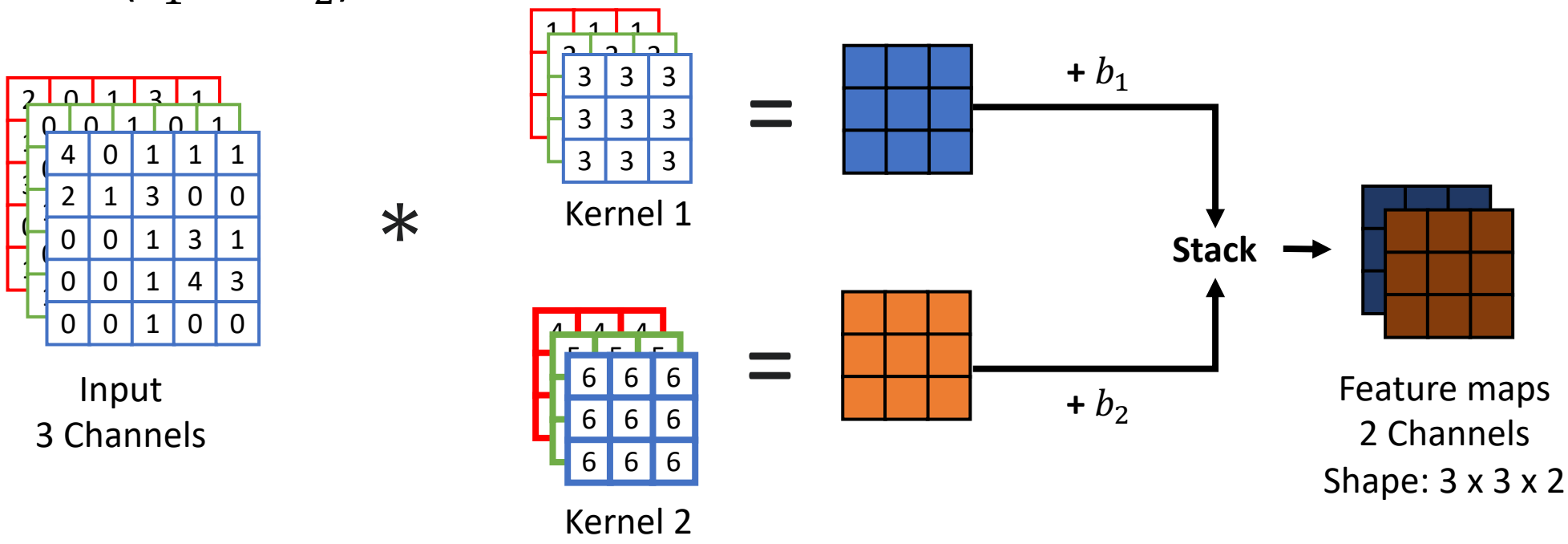
$$Z_i = X * W_i + b_i$$

  - X: The input
  - $*$: The convolution operator
  - W_i: The i-th learnable kernel
  - b_i: Bias, broadcasted and added to every element of the resulting 2D map.

Convolutional Layer: Pre-activation

Example:

- 3 input channels
- 2 output channels
- 2 biases (b_1 and b_2)



Convolutional Layer: Post-activation

Convolution and adding bias are linear operations.

Activation functions are required to introduce non-linearity.

How is it applied?

- The activation function is applied element-by-element to the pre-activation output.
- After activation, the ***i*-th Output Channel** (A_i) is computed as:

$$(A_i)_{p,\,q} = g\big((Z_i)_{p,\,q}\big)$$

Here, $(Z_i)_{p,q}$ is the single scalar value located at row p and column q of the matrix Z_i.

Common activation functions

- ReLU: $g(z) = \max(0, z)$
- Leaky ReLU: $g(z) = \max(az, z)$

Poll Everywhere

PollEV.com/conghuihu365

The setup:

- Input image size: 600×600
- Kernel size: 3×3
- Number of kernels (output channels): 1000

What is the total number of **learnable parameters** in this convolutional layer (with bias)?

A. 9
B. 9,000
C. 10,000
D. 360,000

Convolutional Layer: Parameter Count

The setup (Same example as the FCNN approach):

- Input image size: 600×600
- Kernel size: 3×3
- Number of kernels (output channels): 1000

Learnable parameters in one convolutional layer:

$$\underbrace{3 \times 3 \times 1000}_{\text{Weights}} + \underbrace{1000}_{\text{Bias}} = 10{,}000$$

Convolutional layer reduces parameters by $\sim 36{,}000$ times

Learnable parameters in first fully-connected layer (FCNN approach):

$$\underbrace{360{,}000 \times 1000}_{\text{Weights}} + \underbrace{1000}_{\text{Bias}} = 360{,}001{,}000$$

Convolutional Layer v.s. Fully-connected Layer

	Convolutional Layer	Fully-Connected Layer
Parameter Efficiency	High: Use Fewer parameters	Low: Requires a massive number of parameters
Parameter Sharing	Yes: The same kernel slides across the entire input	No: A different set of weights are used to generate each output value
Spatial Structure	Preserved: Processes 2D/3D inputs directly and produces 2D/3D outputs	Destroyed: Inputs must be flattened into a 1D vector
Connectivity	Local: Each output depends on a small input region	Global: Every neuron connects to all neurons in the previous layer

Typical Roles in CNN

Convolutional Layer: Feature Extractor

- Detect local features, e.g., edges, textures
- Stack multiple convolutional layers: Combine the simple patterns discovered in early layers to form complex features

Fully-Connected Layer: Predictor

- Flatten and combine all features into a global representation
- Produce the final output (e.g., class probabilities)

Computational Overload

The setup for one convolutional layer:

- Input image size: 600×600; $I_H = I_W = 600$
- Kernel size: 3×3; $K = 3$
- Number of kernels (output channels): 1000
- Stride $S = 1$; Padding $P = 1$
- With padding, the height/width of each output feature map:

$$1 + \left\lfloor \frac{I_H + 2P - K}{S} \right\rfloor = 1 + \left\lfloor \frac{600 + 2 - 3}{1} \right\rfloor = 600$$

The number of values in all generated feature maps:

$$\underbrace{600 \times 600}_{\text{One feature map size}} \times \underbrace{1000}_{\text{Number of feature maps}} = 360{,}000{,}000$$

Need to compress these feature maps!

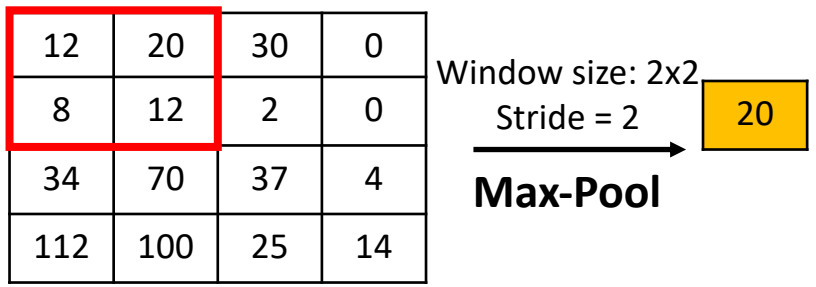
Pooling Layer

How a Pooling Layer Works:

- Sweep a fixed-size sliding window across the input feature map (Step size: Stride)
- Apply a fixed pooling function to condense the items inside the window into a single output value.

Common Pooling Functions:

- **Max-Pool**: Extract the **maximum value** in each window



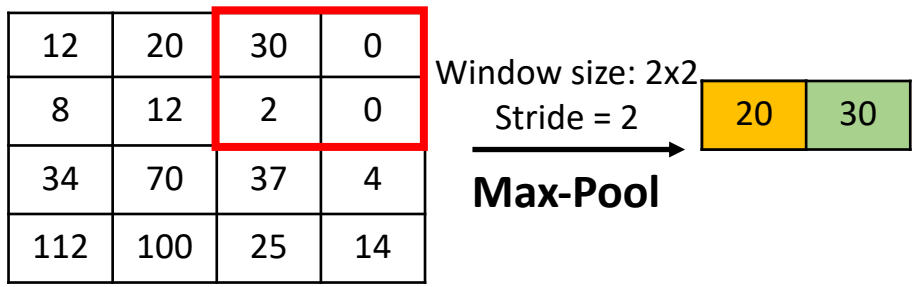
Pooling Layer

How a Pooling Layer Works:

- Sweep a fixed-size sliding window across the input feature map (Step size: Stride)
- Apply a fixed pooling function to condense the items inside the window into a single output value.

Common Pooling Functions:

- **Max-Pool**: Extract the **maximum value** in each window



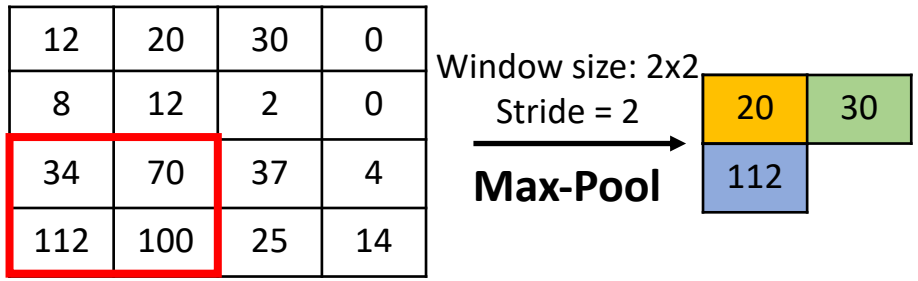
Pooling Layer

How a Pooling Layer Works:

- Sweep a fixed-size sliding window across the input feature map (Step size: Stride)
- Apply a fixed pooling function to condense the items inside the window into a single output value.

Common Pooling Functions:

- **Max-Pool**: Extract the **maximum value** in each window



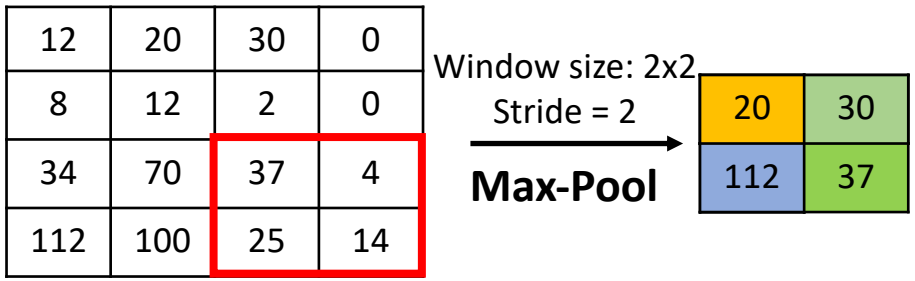
Pooling Layer

How a Pooling Layer Works:

- Sweep a fixed-size sliding window across the input feature map (Step size: Stride)
- Apply a fixed pooling function to condense the items inside the window into a single output value.

Common Pooling Functions:

- **Max-Pool**: Extract the **maximum value** in each window



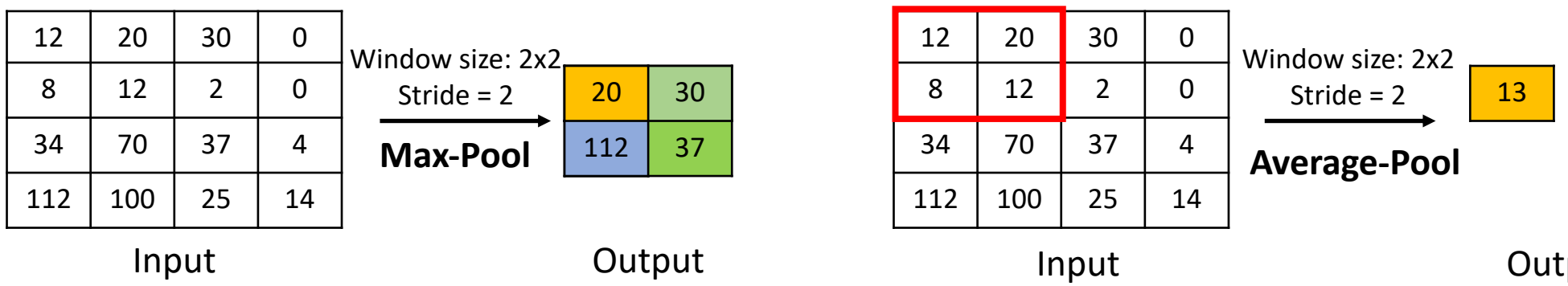
Pooling Layer

How a Pooling Layer Works:

- Sweep a fixed-size sliding window across the input feature map (Step size: Stride)
- Apply a fixed pooling function to condense the items inside the window into a single output value.

Common Pooling Functions:

- **Max-Pool**: Extract the **maximum value** in each window
- **Average-Pool**: Compute the **average value** in each window



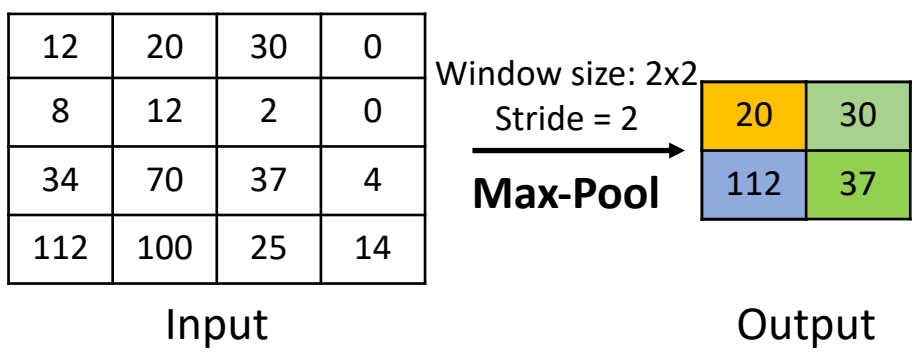
Pooling Layer

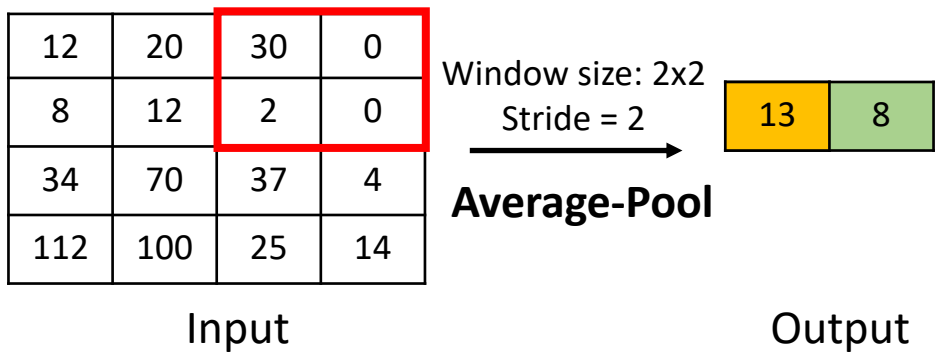
How a Pooling Layer Works:

- Sweep a fixed-size sliding window across the input feature map (Step size: Stride)
- Apply a fixed pooling function to condense the items inside the window into a single output value.

Common Pooling Functions:

- **Max-Pool**: Extract the **maximum value** in each window
- **Average-Pool**: Compute the **average value** in each window





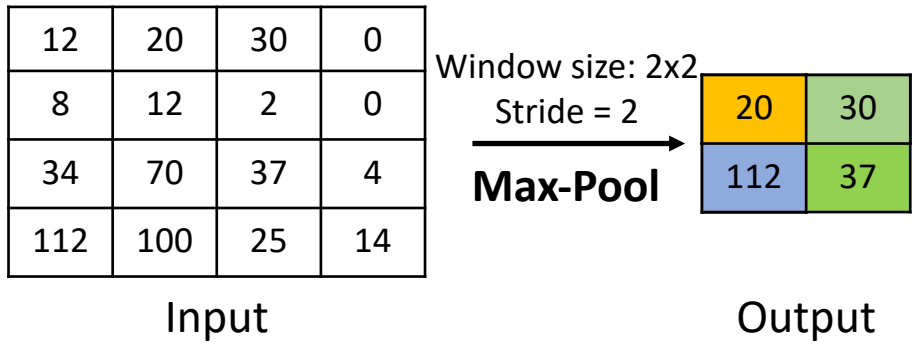
Pooling Layer

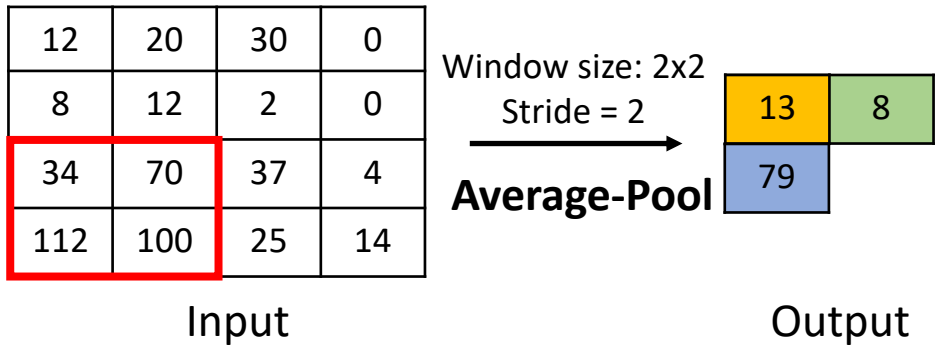
How a Pooling Layer Works:

- Sweep a fixed-size sliding window across the input feature map (Step size: Stride)
- Apply a fixed pooling function to condense the items inside the window into a single output value.

Common Pooling Functions:

- **Max-Pool**: Extract the **maximum value** in each window
- **Average-Pool**: Compute the **average value** in each window





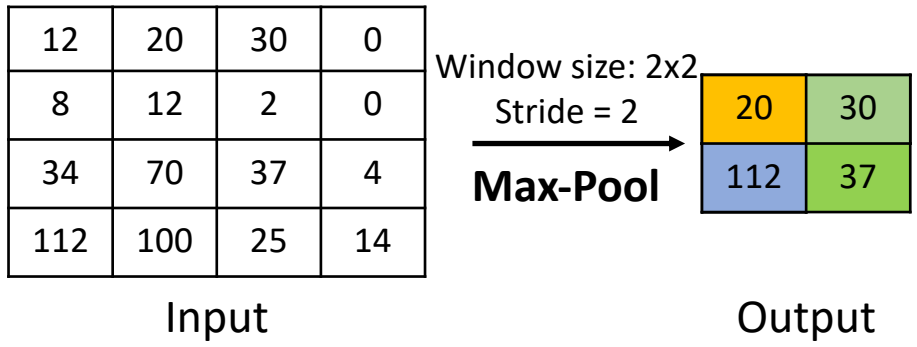
Pooling Layer

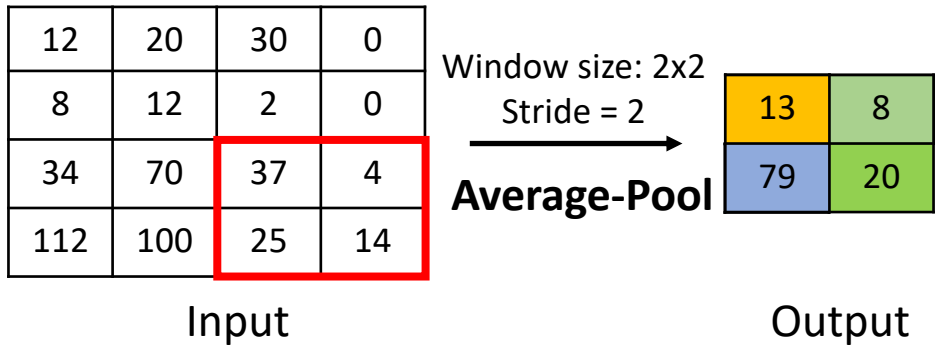
How a Pooling Layer Works:

- Sweep a fixed-size sliding window across the input feature map (Step size: Stride)
- Apply a fixed pooling function to condense the items inside the window into a single output value.

Common Pooling Functions:

- **Max-Pool**: Extract the **maximum value** in each window
- **Average-Pool**: Compute the **average value** in each window



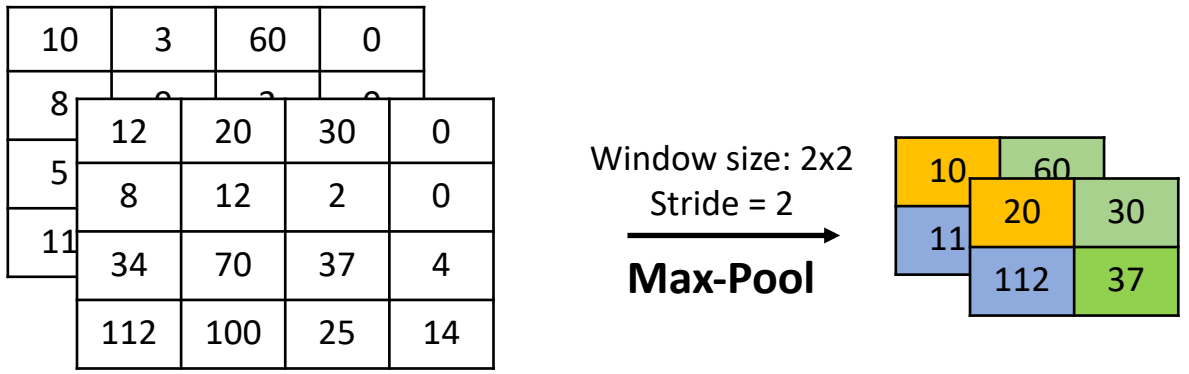


Pooling Layer

Dimensionality Rules:

Spatial downsampling: The sliding window operates across the height and width of the feature map.

Channel independence: Each feature map is processed completely separately from the others.



Input Feature Maps
Shape: 4 x 4 x 2

Output
Shape: 2 x 2 x 2

Typical

Convolutional Neural Network (CNN)

Image Classification

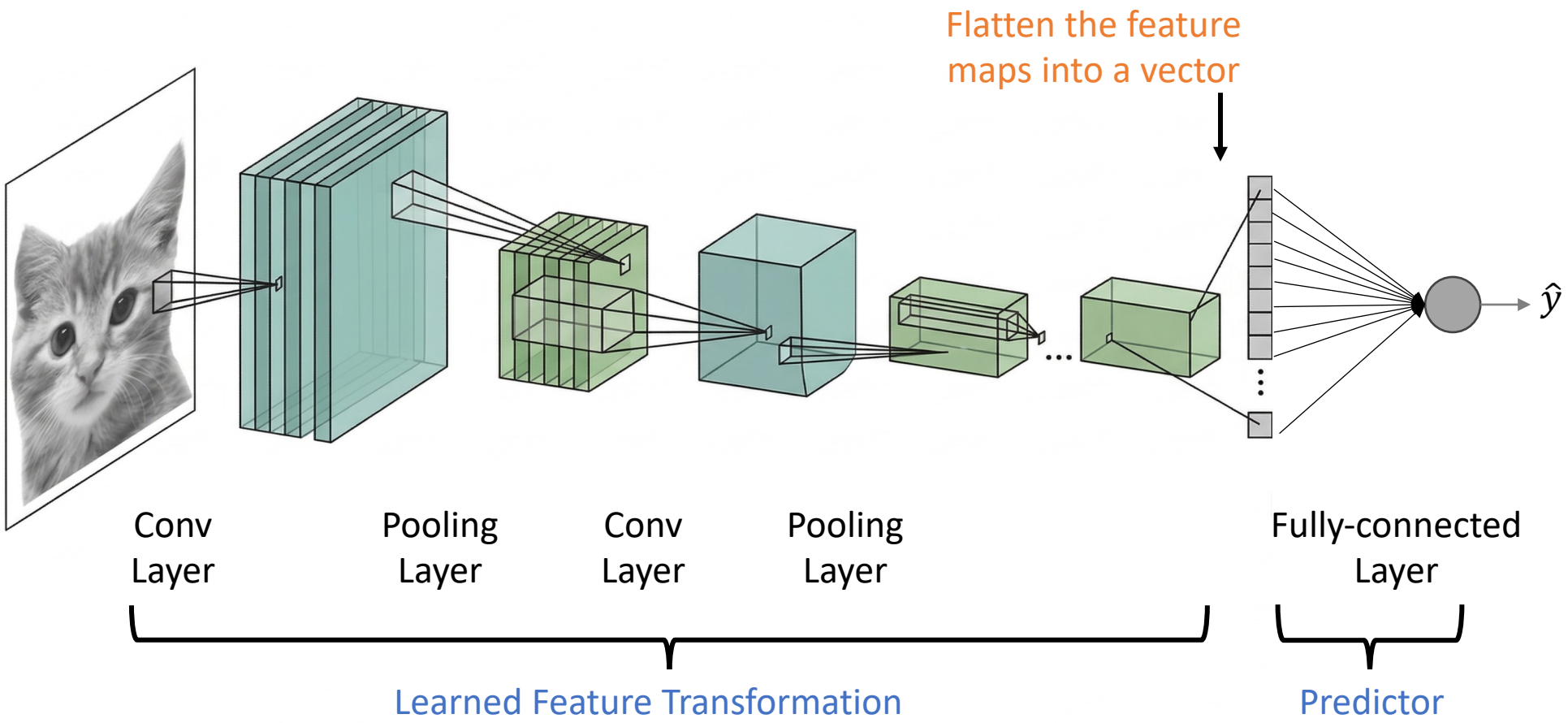


Image credit: https://www.mdpi.com/1424-8220/19/22/4933

Typical

Convolutional Neural Network (CNN)

Image Classification

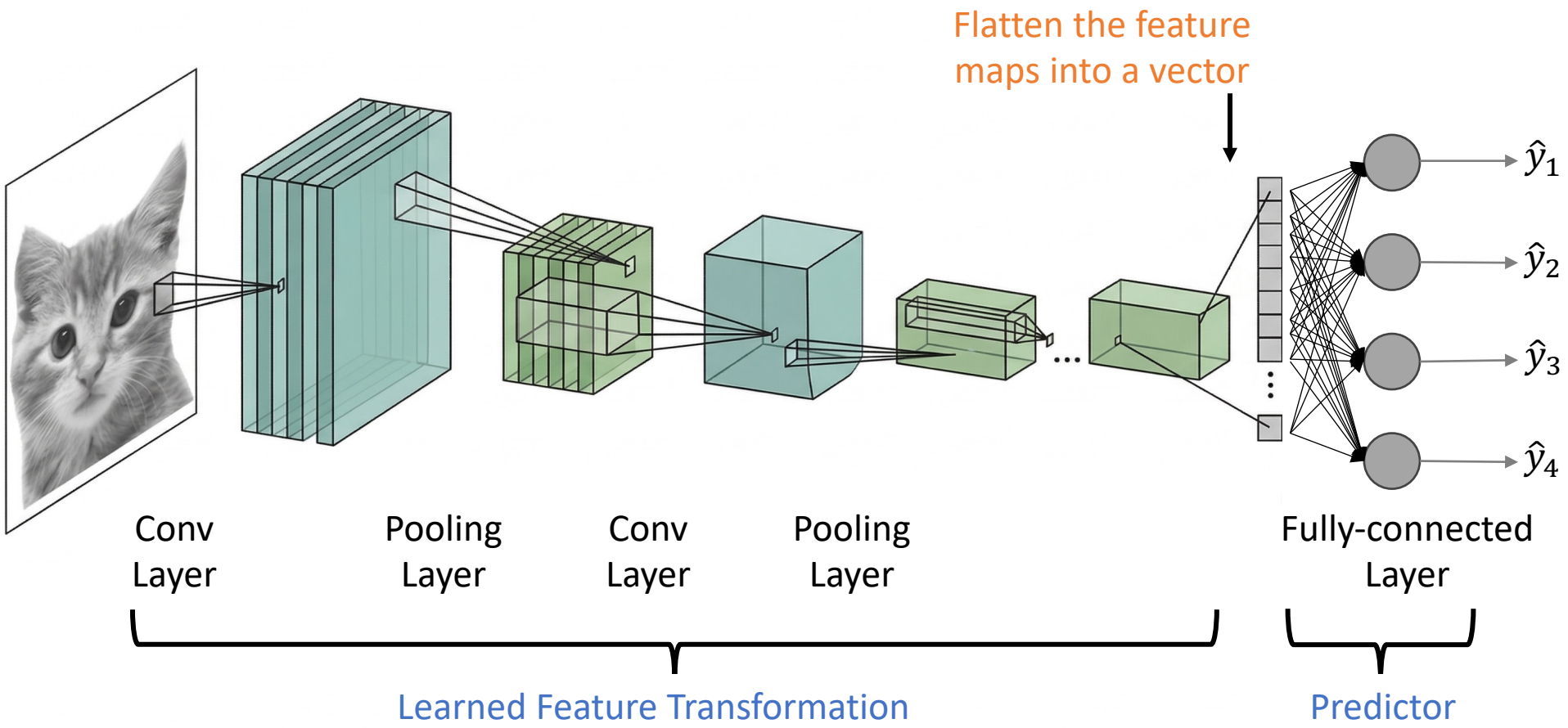
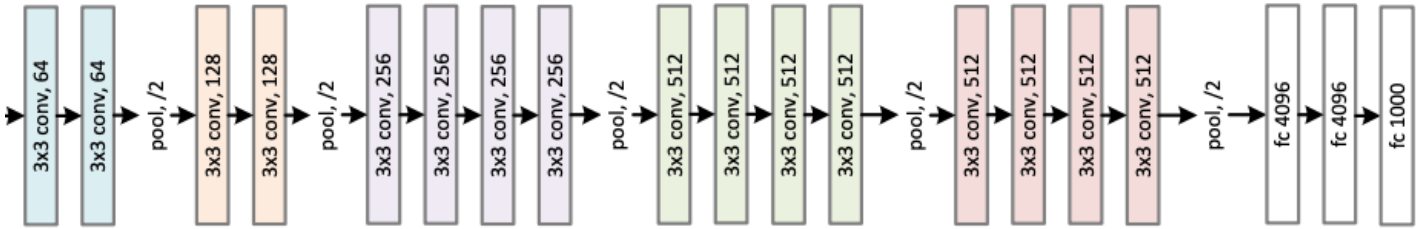


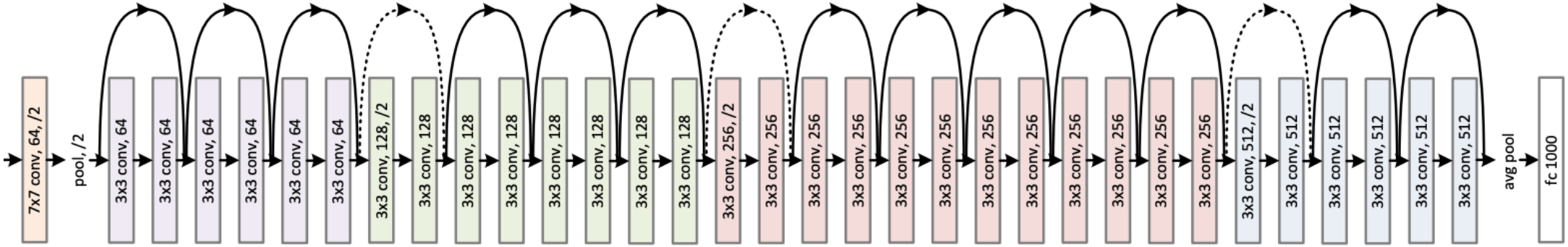
Image credit: https://www.mdpi.com/1424-8220/19/22/4933

Popular CNN Architectures

VGG



ResNet



Further reading: https://www.jeremyjordan.me/convnet-architectures/

Applications of CNN

Image Classification
e.g., face emotions

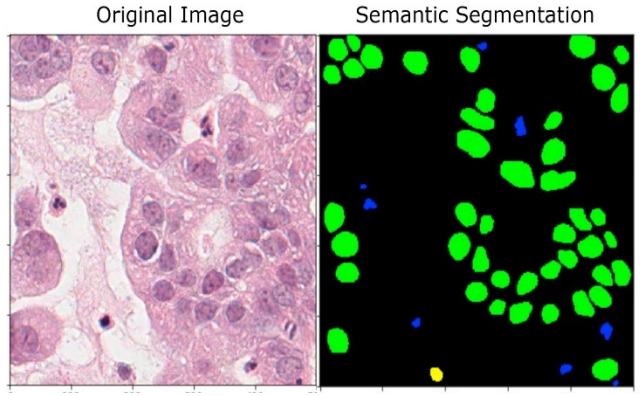
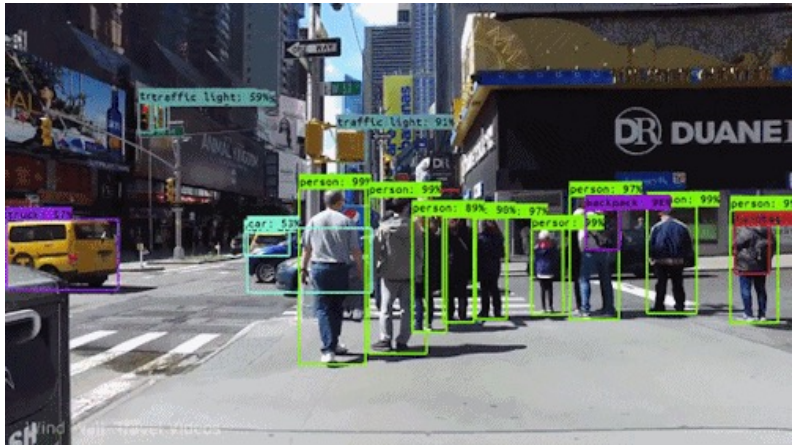


Image Segmentation
e.g., cancer cell detection



Object Detection
e.g., self-driving cars

Image Credit:
https://analyticsindiamag.com/face-emotion-recognizer-in-6-lines-of-code/
https://medium.com/@albertlai631/how-do-self-driving-cars-see-13054aee2503
https://ajp.amjpathol.org/article/S0002-9440(18)31121-0/fulltext

Summary

- FCNN for vision task
  - Parameter Explosion & Loss of Spatial Structure
- Convolution
  - Each kernel can detect a specific type of feature.
  - The number of channels in each kernel must match the number of input channels.
  - The number of kernels determines the number of output channels.
- Convolutional Neural Networks
  - Convolutional Layer: Feature extraction
  - Pooling Layer: Feature map spatial dimension reduction
  - Fully-connected Layer: Prediction

Coming Up Next Week

- **Recurrent Neural Networks**

To Do

- **Lecture Training 9**
  - +550 Free EXP
  - +100 Early bird bonus